

Apuntes Completos

Diseño automático de sistemas fiables
Universidad de Antonio de Nebrija

Marcelo Fort Muñoz

6 de mayo de 2026

Índice general

1. Introducción	1
1.1. Conceptos básicos	1
1.1.1. Historia: la crisis del software	1
1.1.2. Definiciones	1
1.1.3. Conceptos clave	2
1.1.4. Roles en el proceso	2
1.2. Estándares y organizaciones	2
1.2.1. Necesidad de estandarizar	2
1.2.2. Usos de los estándares	2
1.2.3. Organizaciones relevantes	2
1.2.4. Principales estándares ISO	3
1.2.5. SPICE – Niveles de madurez	3
1.2.6. CMMI (Capability Maturity Model Integration)	3
2. El proceso del software	4
2.1. Fases del desarrollo	5
2.1.1. Metodología y procesos	5
2.1.2. Proceso esencial	5
2.1.3. Definiciones	5
2.1.4. Fases del proceso	6
2.1.5. Tipos de flujo de fases	6
2.1.6. Actividades transversales	7
2.2. Modelos del proceso	7
2.2.1. Modelo cascada: Versión V	8
2.2.2. Modelo de proceso incremental	9
2.2.3. Proceso evolutivo	10
2.2.4. Modelo Concurrente	11
2.2.5. Proceso unificado	12
2.2.6. Otros modelos	12
2.2.7. Ciclo de vida del producto	12
2.3. Desarrollo ágil	13
2.3.1. Definiciones	13
2.3.2. El coste del cambio	13
2.3.3. Características de los métodos ágiles	13
2.3.4. Manifiesto ÁGIL	13
2.3.5. Principios del Manifiesto Ágil	13
2.3.6. Roles en el desarrollo ágil	14
2.3.7. Programación extrema (XP)	14
2.3.8. SCRUM	14
2.3.9. KANBAN	15

2.3.10. Otras metodologías ágiles	16
3. Modelado del software	17
3.1. Requisitos del software	18
3.1.1. Principios de comunicación	18
3.1.2. Definición de requisito	18
3.1.3. Tipos de Requisitos	18
3.1.4. Clasificación de requisitos no funcionales	19
3.1.5. Dificultades Comunes	20
3.1.6. Ingeniería de requisitos	21
3.1.7. Modelado de requisitos	24
3.2. Diseño del software	25
3.2.1. Definición y propósito	25
3.2.2. Importancia del diseño	25
3.2.3. Atributos de calidad del software	25
3.2.4. Conceptos clave del diseño	25
3.2.5. Clases de diseño	26
3.2.6. Elementos del modelo de diseño	26
3.2.7. Diseño de arquitectura y patrones	26
3.2.8. Diseño de interfaz de usuario	28
3.2.9. Componentes	29
3.2.10. Revisión técnica formal	29
4. Planificación de proyectos	31
4.1. Estimación	32
4.1.1. Objetivo y proceso de la planificación	32
4.1.2. Ámbito vs Factibilidad	32
4.1.3. Técnicas de Estimación	33
4.1.4. Técnicas de Descomposición	33
4.1.5. COCOMO II	34
4.1.6. Estimación Ágil	35
4.2. Calendarización	35
4.2.1. Relación con la Estimación	35
4.2.2. Dificultades Comunes	36
4.2.3. Manejo de Fechas Límite	36
4.2.4. Hitos y Entregas	36
4.2.5. Dependencias entre Tareas	37
4.2.6. Diagrama de Gantt	37
4.2.7. Diagrama de PERT	37
4.3. Administración del Riesgo	38
4.3.1. Clasificación de Riesgos	39
4.3.2. Descripción de Riesgos	39
4.3.3. Estrategias de Gestión de Riesgos	40
4.3.4. Proceso de Gestión de Riesgos	41
4.3.5. Actividades por fase	41
5. Calidad	42
5.1. Aseguramiento de la Calidad	43
5.1.1. Principios rectores de la calidad	43
5.1.2. Atributos de las métricas	43
5.1.3. Control vs. Aseguramiento de la Calidad	44
5.1.4. Revisiones técnicas	44

5.1.5.	Revisiones durante el desarrollo	44
5.1.6.	Métricas de calidad (DORA Metrics)	44
5.1.7.	Buenas prácticas de desarrollo	45
5.1.8.	Métricas de desarrollo	45
5.1.9.	Deuda técnica	45
5.2.	Estrategias de Prueba	46
5.2.1.	Verificación vs. Validación	46
5.2.2.	Malas prácticas	46
5.2.3.	Estrategias de prueba (pirámide de pruebas)	46
5.2.4.	Dimensiones de la prueba	46
5.2.5.	Pruebas específicas	46
5.3.	Administración de la Configuración	47
5.3.1.	Ítems de Configuración (IC)	47
5.3.2.	Sistema de Administración	47
5.3.3.	Repositorio ACS	48
5.3.4.	Proceso de Administración de la Configuración	48
5.3.5.	Administración del contenido	48
5.3.6.	Conflictos dominantes	48
6.	Mantenimiento	49
6.1.	Métricas de producto	49
6.1.1.	Conceptos clave	49
6.1.2.	Principios de medición	49
6.1.3.	Atributos de métricas	50
6.1.4.	Métricas específicas	50
6.2.	Tipos de mantenimiento	52
6.2.1.	Leyes de Lehman	52
6.2.2.	Proceso de mantenimiento	52
6.2.3.	Clasificación de mantenimiento	53
6.2.4.	Release Notes	53
6.2.5.	Sistemas heredados	53
6.2.6.	Reingeniería de sistemas	53
6.2.7.	Tipos de reingeniería	53
6.2.8.	Flujo de reingeniería	54
Anexos		55
Anexo 1:	Clasificación de procesos según ISO/IEC 15504	56
Anexo 2:	Explicación de la Administración de Configuración de Software (ACS)	57

Capítulo 1

Introducción

Índice general

1.1. Conceptos básicos	1
1.1.1. Historia: la crisis del software	1
1.1.2. Definiciones	1
1.1.3. Conceptos clave	2
1.1.4. Roles en el proceso	2
1.2. Estándares y organizaciones	2
1.2.1. Necesidad de estandarizar	2
1.2.2. Usos de los estándares	2
1.2.3. Organizaciones relevantes	2
1.2.4. Principales estándares ISO	3
1.2.5. SPICE – Niveles de madurez	3
1.2.6. CMMI (Capability Maturity Model Integration)	3

1.1. Conceptos básicos

1.1.1. Historia: la crisis del software

- Término acuñado en 1968 en una conferencia de la OTAN.
- Refleja la dificultad para desarrollar software útil y eficiente en tiempo y forma.
- Ejemplos de grandes fracasos:
 - **Mariner 1 (1962):** pérdida de 18.500.000\$.
 - **Therac-25 (1982):** 3 fallecidos y 3 con secuelas por errores de software.
 - **Caída AT&T (1990):** 75 millones de llamadas afectadas.
 - **Knight Capital (2012):** pérdida de 500.000.000\$.

1.1.2. Definiciones

Definición

Software:

- Instrucciones que al ejecutarse proporcionan funciones, características y rendimiento.
- Estructuras de datos que permiten manipular información adecuadamente.

Definición

Ingeniería del Software (IEEE, 1993): Aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento del software.

Definición

Ingeniería del Software (Fritz Bauer, 1969): Uso de principios fundamentales de la ingeniería para desarrollar software fiable y eficiente de forma económica.

1.1.3. Conceptos clave

- **Personas:** Creadores del producto.
- **Producto:** Resultado final del desarrollo.
- **Usuarios:** Receptores del producto.
- **Proyecto:** Conjunto de hitos hacia un resultado.
- **Proceso:** Fases del desarrollo del software.
- **Ciclo de vida:** Evoluciones del producto en el tiempo.

1.1.4. Roles en el proceso

- **Gestor de producto:** Define funcionalidades y prioridades.
- **Gestor de proyecto:** Asegura recursos para cumplir el plan.
- **Ingeniero de software:** Diseña e implementa el software.
- **Ingeniero de calidad:** Verifica el correcto funcionamiento del producto.

1.2. Estándares y organizaciones

1.2.1. Necesidad de estandarizar

- **Proceso tipo:** Marco para valorar e identificar mejoras.
- **Buenas prácticas:** Lo que funciona se institucionaliza.
- **Lenguaje común:** Mejora la comunicación entre roles.

1.2.2. Usos de los estándares

- **Certificación:** Requisito para:
 - Acceder a proyectos regulados.
 - Comercializar productos y servicios.

1.2.3. Organizaciones relevantes

- **IEEE:** Institute of Electrical and Electronics Engineers (global).
- **SEI:** Software Engineering Institute (EUA).
- **ISO:** International Organization for Standardization (global).

- **IEC:** International Electrotechnical Commission (global).
- **EIA:** Electronic Industries Alliance (EUA).

1.2.4. Principales estándares ISO

- **ISO 9126:** Evalúa la calidad del software en:
 - Funcionalidad, fiabilidad, usabilidad, eficiencia, mantenibilidad, portabilidad y satisfacción.
- **Familia ISO 9000:** Gestión de calidad
 - **ISO 9000:** Fundamentos y vocabulario.
 - **ISO 9001:** Requisitos de calidad.
 - **ISO 9004:** Mejora continua.
- **ISO/IEC 12207:** Procesos del ciclo de vida del software: procesos principales, de apoyo y organizativos.
- **ISO/IEC 15504 (SPICE):** Evalúa y mejora procesos de ingeniería del software.

1.2.5. SPICE – Niveles de madurez

Nivel 0 - Incompleto: Sin implementación efectiva.

Nivel 1 - Realizado: Procesos implementados y objetivos alcanzados.

Nivel 2 - Gestionado: Procesos y productos controlados.

Nivel 3 - Establecido: Procesos basados en estándares.

Nivel 4 - Predecible: Gestión cuantitativa con objetivos.

Nivel 5 - Optimizado: Mejora continua y búsqueda de buenas prácticas.

1.2.6. CMMI (Capability Maturity Model Integration)

- Establecido por el SEI.
- Actualmente gestionado por el CMMI Institute.
- Basado en SPICE, muy popular en EUA.

Capítulo 2

El proceso del software

Índice general

2.1.	Fases del desarrollo	5
2.1.1.	Metodología y procesos	5
2.1.2.	Proceso esencial	5
2.1.3.	Definiciones	5
2.1.4.	Fases del proceso	6
2.1.5.	Tipos de flujo de fases	6
	Lineal	6
	Iterativo	6
	Evolutivo	6
	Paralelo	7
2.1.6.	Actividades transversales	7
2.2.	Modelos del proceso	7
2.2.1.	Modelo cascada: Versión V	8
2.2.2.	Modelo de proceso incremental	9
2.2.3.	Proceso evolutivo	10
2.2.4.	Modelo Concurrente	11
2.2.5.	Proceso unificado	12
2.2.6.	Otros modelos	12
	Modelos especializados	12
	Modelos personales y de equipo	12
2.2.7.	Ciclo de vida del producto	12
2.3.	Desarrollo ágil	13
2.3.1.	Definiciones	13
2.3.2.	El coste del cambio	13
2.3.3.	Características de los métodos ágiles	13
2.3.4.	Manifiesto ÁGIL	13
2.3.5.	Principios del Manifiesto Ágil	13
2.3.6.	Roles en el desarrollo ágil	14
2.3.7.	Programación extrema (XP)	14
2.3.8.	SCRUM	14
	Ceremonias SCRUM	14
	Artefactos SCRUM	15
2.3.9.	KANBAN	15
	Principios KANBAN:	15
	Prácticas KANBAN:	16

2.3.10. Otras metodologías ágiles	16
---	----

2.1. Fases del desarrollo

2.1.1. Metodología y procesos

Para el desarrollo exitoso de un software se requiere de ciertos «componentes» que permiten que la «máquina» funcione sin «atascarse». Estos «engranajes» son:

1. Herramientas: Tanto control de versiones (git, subversion,...), como IDEs y otros **programas o dispositivos** que ayudan al desarrollo.
2. Métodos: **Técnicas** usadas para facilitar el desarrollo.
3. Proceso: **Actividades, acciones y tareas** realizadas con el fin de crear productos.

2.1.2. Proceso esencial

En un desarrollo habitual, el proceso suele ser el siguiente:

1. **Entender el problema:** Comunicación y análisis (participantes, incógnitas).
2. **Planear la solución:** Modelado y diseño (descomposición del problema).
3. **Ejecutar el plan:** Creación de código (seguimiento y revisión).
4. **Examinar el resultado:** Pruebas y aseguramiento de la calidad.

2.1.3. Definiciones

Definición

Un **proceso** es aquel **conjunto de actividades, acciones y tareas** que se ejecutan cuando va a crearse algún producto del trabajo.

Definición

Una **actividad** busca lograr un **objetivo amplio** sin importar el dominio de la aplicación, tamaño del proyecto, complejidad del esfuerzo.

Definición

Una **acción** es un **conjunto de tareas** que producen un producto importante del trabajo.

Definición

Una **tarea** se centra en un objetivo **pequeño pero bien definido** que produce un **resultado tangible**.

Definición

Un **proceso** no es una prescripción rígida. Es un enfoque **adaptable** que permite entregar el software de **forma oportuna** y con **calidad suficiente**.

2.1.4. Fases del proceso

El proceso de ingeniería del software se divide en 5 sencillas fases: **Comunicación**, **planificación**, **diseño**, **desarrollo** y, **despliegue**. Se puede observar que es y que objetivo tiene cada fase en la Tabla 2.1

Fase	Objetivo	Detalles
Comunicación	Entender requisitos.	Colaboración con participantes, definición de características.
Planificación	Definir el trabajo	Tareas técnicas, riesgos, recursos, cronograma.
Diseño	Plantear solución	Diagramas de alto nivel, refinamiento iterativo.
Desarrollo	Construir y verificar	Programación y pruebas.
Despliegue	Entregar producto	Evaluación y <i>feedback</i> .

Cuadro 2.1: Comparación de las distintas fases del proceso de desarrollo de software

2.1.5. Tipos de flujo de fases

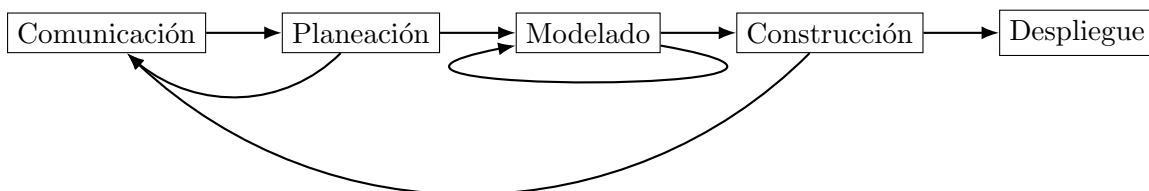
Existen varios tipos de flujo de fases. A continuación, varios diagramas con los distintos flujos:

Lineal

De una en una, sencillo.

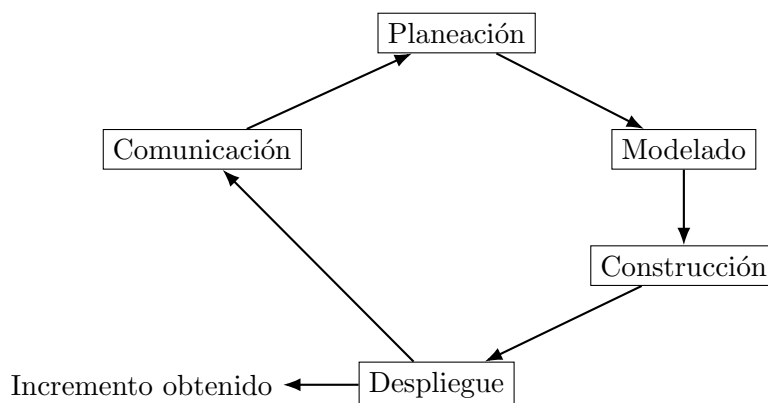


Iterativo



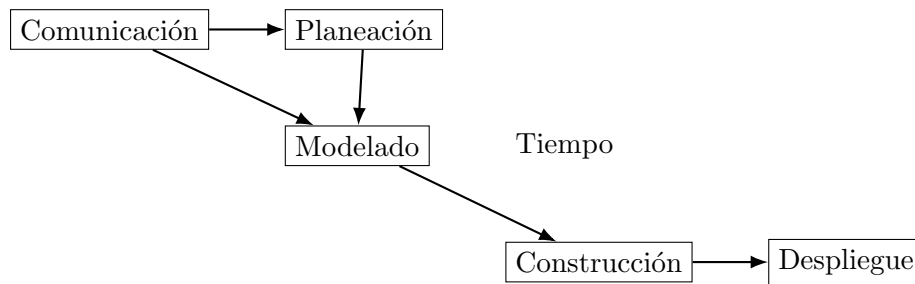
Evolutivo

Permite un prototipado rápido. Resiliente y con una planificación adaptativa.



Paralelo

Fases en paralelo para optimizar la eficiencia



2.1.6. Actividades transversales

Las siguientes tareas se llevan a cabo de forma mas o menos continua durante la duración total del proyecto:

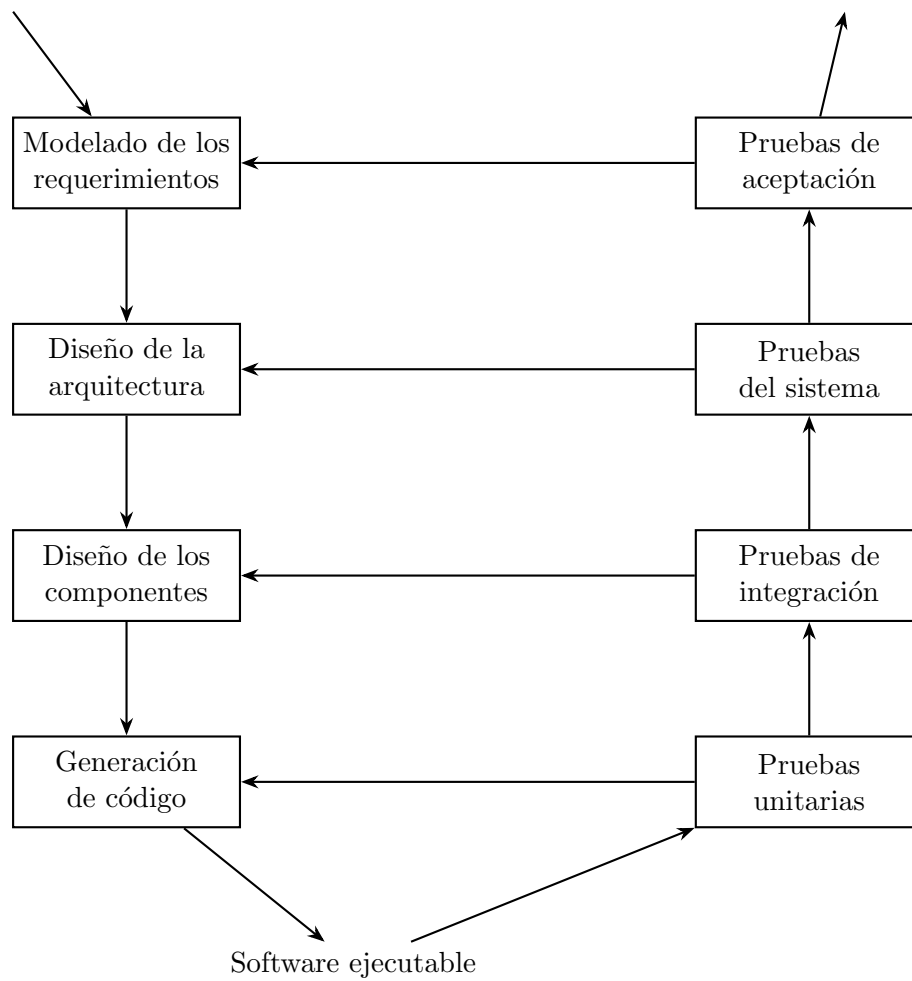
- **Seguimiento y control:** Evaluación del estado del proyecto con respecto al plan original.
- **Administración de riesgos:** Identificación y mitigación.
- **Seguimiento de calidad del software:** Revisiones y estándares que **garantizan** que el software tiene la calidad prometida.
- **Medición:** Métricas de proceso, proyecto y producto.
- **Gestión de la configuración:** Control de cambios.
- **Reutilización:** Creación e uso de componentes reutilizables.
- **Preparación y producción del producto del trabajo:** Actividades para crear productos del trabajo, tales como modelos y documentación.

2.2. Modelos del proceso

No todos los proyectos son iguales. En este tema veremos los modelos de procesos mas usados.

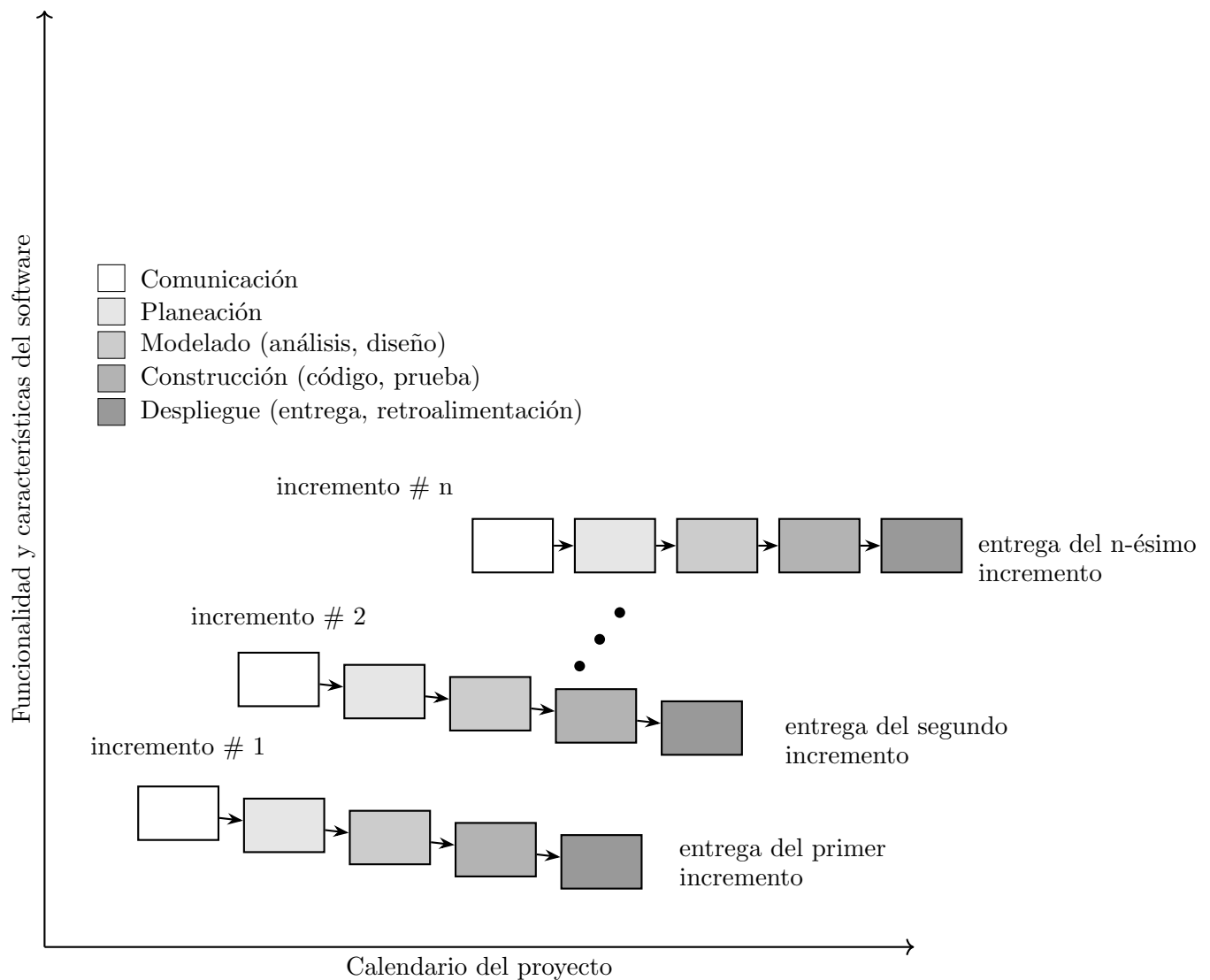
2.2.1. Modelo cascada: Versión V

Tiene fases secuenciales y lineales. Las pruebas y el desarrollo funcionan en paralelo, de ahí la forma de V.



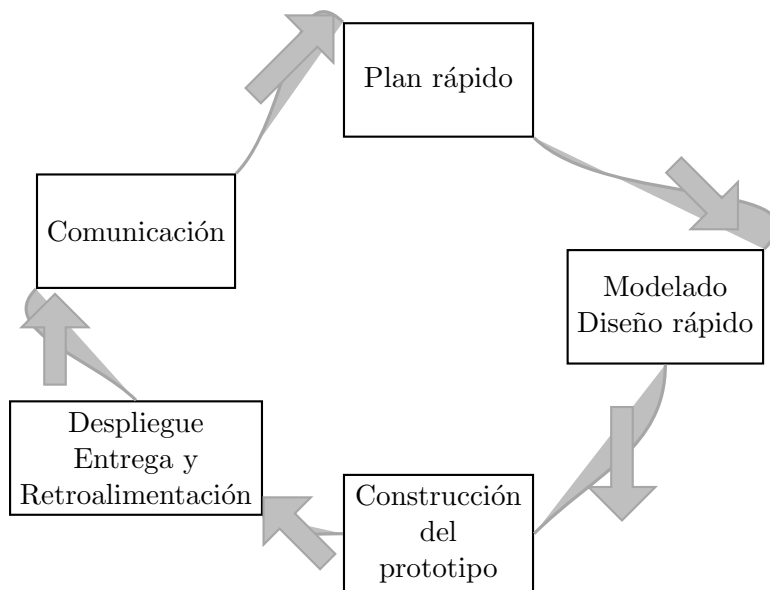
2.2.2. Modelo de proceso incremental

El proceso se divide en incrementos que incluyen todas las fases. Esto implica que todas las entregas intermedias (incremento x) aseguran un cierto nivel de calidad.



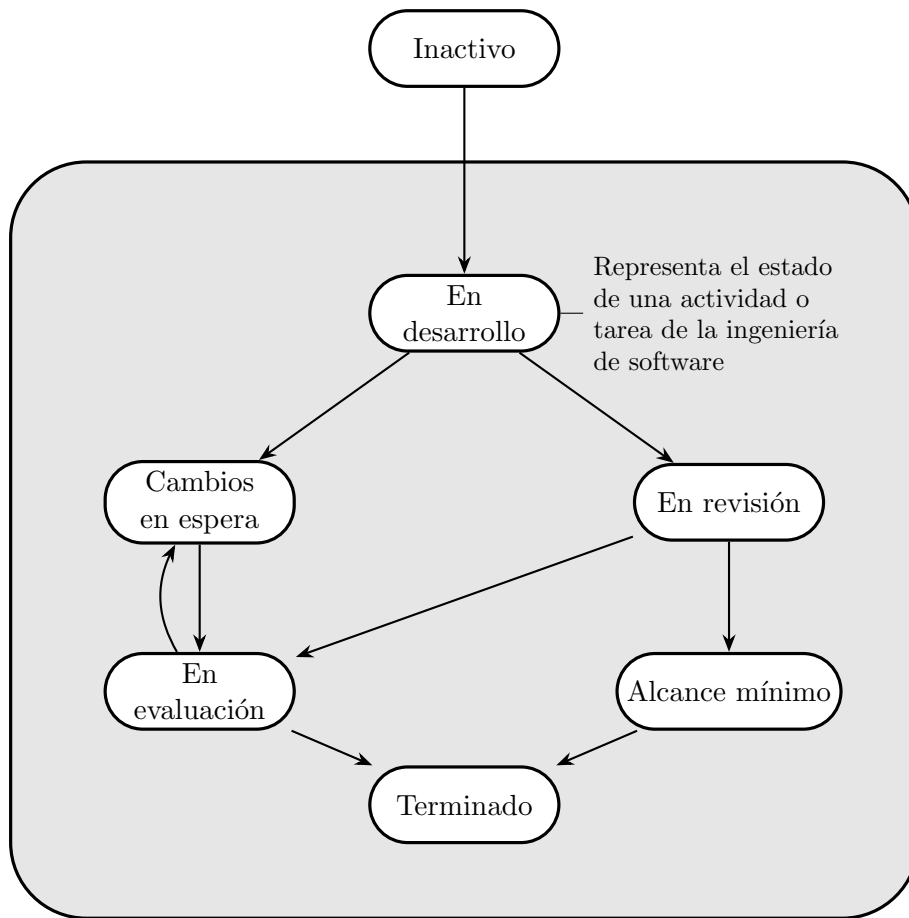
2.2.3. Proceso evolutivo

Con forma de «espiral», permite adaptar los requisitos y solucionar los problemas de iteraciones antiguas.



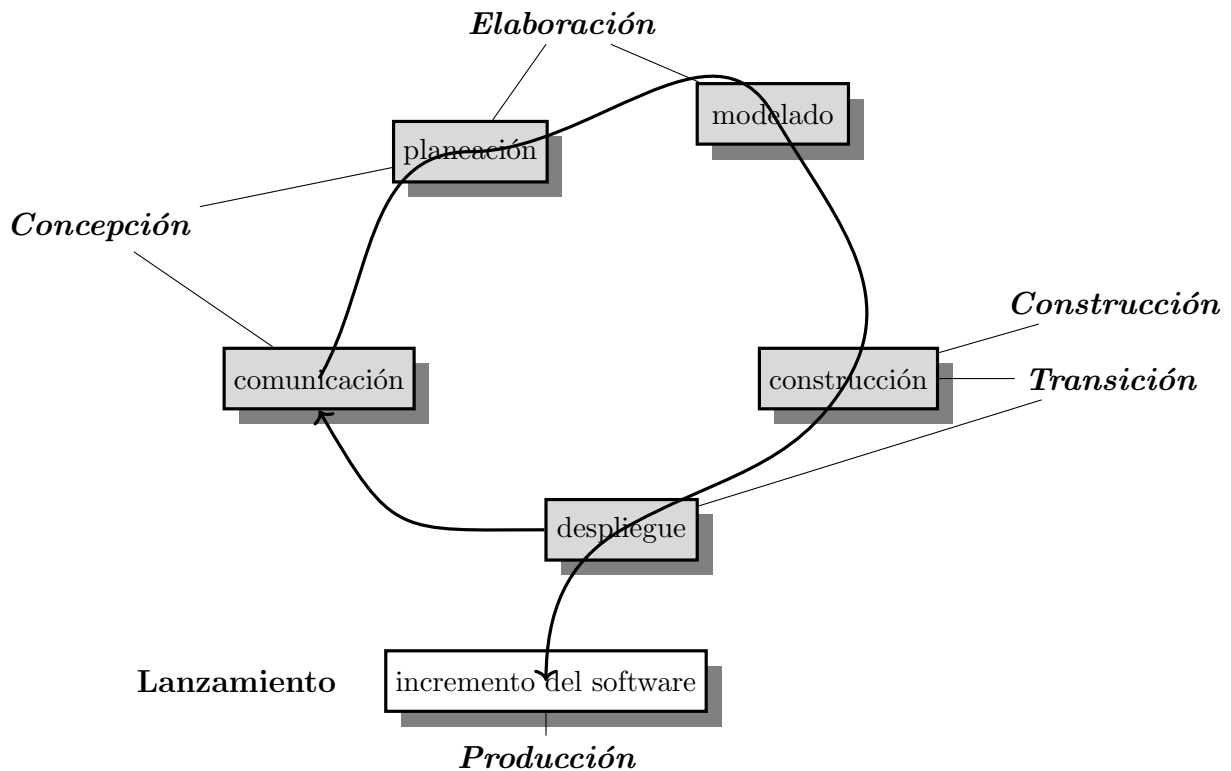
2.2.4. Modelo Concurrente

Funciona con el diseño y el desarrollo en paralelo. Muy versátil.



2.2.5. Proceso unificado

Compuesta por fases múltiples con ciclos cortos.



2.2.6. Otros modelos

Modelos especializados

- **Desarrollo basado en componentes:** utiliza un enfoque evolutivo y iterativo
- **Modelo de métodos formales:** basado en modelos matemáticos para especificación rigurosa
- **Desarrollo orientado a aspectos:** combina enfoques evolutivos y concurrentes

Modelos personales y de equipo

- **Proceso Personal del Software (PPS):** metodología individual que incluye:
 - Planificación
 - Diseño
 - Revisión
 - Desarrollo
 - Post mórtem
- **Proceso del Equipo de Software (PES):** equipos auto-dirigidos que se gestionan autónomamente.

2.2.7. Ciclo de vida del producto

Etapas: Introducción → Crecemento → Madurez → Declive.

2.3. Desarrollo ágil

2.3.1. Definiciones

Definición

Agilidad: Capacidad de **adaptación al cambio** (requisitos, equipo, tecnologías,...)

Definición

Agilismo: Métodos para alcanzar agilidad.

2.3.2. El coste del cambio

Los métodos ágiles buscan reducir el coste del cambio a lo largo del ciclo de vida del proyecto, evitando que aumente exponencialmente en las fases tardías.

2.3.3. Características de los métodos ágiles

- **Compatibles con estándares:** CMMI, ISO, etc.
- **Diferenciación:** Los estándares indican el QUÉ hacer, el agilismo indica el CÓMO hacerlo
- **Modelos iterativos y adaptativos:** pueden ser incrementales o evolutivos
- **Equipos multidisciplinares:** autónomos y autoorganizados.
- **Guiados por el Manifiesto Ágil** (2001)
- **Múltiples metodologías:** adopción flexible según necesidades

2.3.4. Manifiesto ÁGIL

Cuatro valores fundamentales (priorizando los de la izquierda sobre los de la derecha):

1. **Individuos e interacciones** sobre procesos y herramientas
2. **Software funcionando** sobre documentación extensiva
3. **Colaboración con el cliente** sobre negociación contractual
4. **Respuesta ante el cambio** sobre seguir un plan

2.3.5. Principios del Manifiesto Ágil

1. **Prioridad:** Satisfacer al cliente mediante entrega temprana y continua de software con valor.
2. **Cambios:** Aceptar que los requisitos cambien, incluso en etapas tardías.
3. **Entregas frecuentes:** Software funcional cada 2 semanas a 2 meses (preferiblemente más corto).
4. **Colaboración diaria:** Responsables de negocio y desarrolladores trabajan juntos.
5. **Individuos motivados:** Dar entorno y apoyo, confiar en la ejecución.
6. **Comunicación cara a cara:** Método más eficiente y efectivo.
7. **Software funcionando:** Principal medida de progreso.
8. **Desarrollo sostenible:** Mantener ritmo constante indefinidamente.

9. **Excelencia técnica:** Atención continua al buen diseño mejora la agilidad.
10. **Simplicidad:** Arte de maximizar la cantidad de trabajo no realizado.
11. **Equipos auto-organizados:** Las mejores arquitecturas, requisitos y diseños emergen de ellos.
12. **Reflexión regular:** El equipo reflexiona para ser más efectivo y ajustar comportamiento.

2.3.6. Roles en el desarrollo ágil

- **Agile Coach:** Experto en agilismo que ayuda a los empleados a adoptar metodologías ágiles
- **Product Owner:** Gestor de la pila de trabajo y su prioridad para maximizar valor entregado
- **Scrum Master:** Facilitador de los equipos que siguen metodología Scrum
- **Equipo de desarrollo:** Conjunto de miembros que desarrollan y entregan software en incrementos de valor

2.3.7. Programación extrema (XP)

La Programación Extrema fue creada por Kent Beck, quien también fue contribuidor al manifiesto ágil. Se caracteriza por llevar las buenas prácticas de programación a sus límites extremos.

Busca **retroalimentación continua** del usuario mediante entregas cortas y frecuentes, lo que permite **detectar y corregir problemas rápidamente**. La **documentación es simple** y se basa en tres principios: mantener **código simple y mantenible**, priorizar **código autodo-cumentado** sobre comentarios extensos, y usar **tests unitarios** como mecanismo de diseño y documentación.

La **programación por parejas** implica que dos desarrolladores trabajen juntos en el mismo código, lo que mejora la calidad y facilita la transferencia de conocimiento. El **énfasis en pruebas** se materializa a través del Test Driven Development (TDD), donde las pruebas se escriben antes que el código, y se eliminan defectos antes de añadir nueva funcionalidad.

El principio **YAGNI** ("You Aren't Gonna Need It") promueve programar solo para las prioridades inmediatas, evitando la sobreingeniería y el desarrollo de funcionalidades que podrían no ser necesarias.

2.3.8. SCRUM

Scrum fue concebido a principios de la década de 1990 por Jeff Sutherland, otro contribuidor al manifiesto ágil. Se estructura en **iteraciones de 2 a 4 semanas** de duración, donde cada iteración debe terminar con una entrega de valor tangible al cliente.

Una característica fundamental es que **el alcance no puede modificarse** durante el desarrollo de la iteración, lo que proporciona estabilidad al equipo. Incorpora un **proceso de mejora continua** para incrementar la eficiencia del equipo mediante retrospectivas regulares.

Ceremonias SCRUM

El **Sprint Planning** es la sesión de planificación donde el equipo decide qué elementos del product backlog se desarrollarán en la siguiente iteración, basándose en la priorización establecida por el Product Owner.

El **Daily Scrum** es una reunión diaria de máximo 15 minutos donde cada miembro del equipo comparte tres elementos: el **progreso realizado desde la anterior reunión**, el **progreso esperado hasta la siguiente reunión**, y **cualquier bloqueo o impedimento que esté enfrentando**.

El **Sprint Review** es la sesión donde se revisa la iteración completada mediante una demostración del software funcional desarrollado.

El **Refinamiento** son sesiones dedicadas a revisar y clarificar requisitos para alcanzar un entendimiento común entre todos los miembros del equipo.

La **Retrospectiva** es una sesión de revisión del proceso utilizado durante la iteración, donde el equipo identifica qué funcionó bien, qué puede mejorarse, y define acciones concretas de mejora.

Artefactos SCRUM

El **Product Backlog** es la pila de trabajo que contiene todos los requisitos y funcionalidades a cumplir, ordenados por prioridad y valor de negocio.

El **Sprint Backlog** contiene específicamente los requisitos que se van a desarrollar en la iteración próxima, con el nivel de detalle necesario para su implementación.

El **Scrum Board** es un tablero visual que muestra el estado actual de todas las tareas, típicamente organizado en columnas como «Por hacer», «En progreso» y «Terminado».

El **Burndown Chart** es un gráfico que compara el progreso ideal de una iteración con el progreso real, permitiendo identificar desviaciones y tomar medidas correctivas.

La **Definition of Ready (DoR)** establece los criterios que debe cumplir una tarea para estar lista para ser iniciada por el equipo de desarrollo.

La **Definition of Done (DoD)** define los criterios que debe cumplir una tarea para poder ser marcada como completada y entregada.

2.3.9. KANBAN

Kanban fue definido por primera vez en 2007 y está basado en el proceso de gestión visual desarrollado por Toyota para la manufactura. Sigue el proceso **Kaizen** de mejora continua y se caracteriza por **no tener iteraciones ni ceremonias preestablecidas**, siendo **más flexible en su estructura**.

Principios KANBAN:

Los **principios de gestión del cambio** establecen que se debe comenzar con lo que se hace actualmente (sin cambios disruptivos), aceptar el cambio incremental y evolutivo (evitando transformaciones radicales), y fomentar actos de liderazgo a todos los niveles de la organización:

- Comienza con lo que haces ahora
- Aceptar el cambio incremental y evolutivo
- Fomentar los actos de liderazgo a todos los niveles

Los **principios de prestación de servicios** se centran en las necesidades y expectativas del cliente como foco principal, gestionar el trabajo y los procesos en lugar de microgestionar a los trabajadores, y revisar periódicamente toda la red de servicios para optimizar el flujo de valor:

- Centrarte en las necesidades y expectativas del cliente

- Gestionar el trabajo, no los trabajadores
- Revisar periódicamente la red de servicios

Prácticas KANBAN:

Visualizar el flujo de trabajo mediante tableros que muestren claramente el estado de todas las tareas y su progreso a través del proceso.

Limitar el trabajo en curso (WIP - Work In Progress) para evitar la sobrecarga del sistema y mejorar el flujo.

Gestionar el flujo monitorizando y optimizando el movimiento del trabajo a través del sistema.

Explicitar las políticas de procesos para que todos entiendan claramente cómo funciona el sistema.

Aplicar bucles de retroalimentación para obtener información sobre el rendimiento del sistema y áreas de mejora.

Mejorar en colaboración mediante el trabajo conjunto de todo el equipo para optimizar el sistema.

2.3.10. Otras metodologías ágiles

Lean Software Development (LSD) está basado en los principios del Lean Startup y se enfoca en eliminar desperdicios, amplificar el aprendizaje y entregar valor rápidamente.

Desarrollo Adaptativo de Software (DAS) es un enfoque que asume que los proyectos de software son inherentemente impredecibles y se adapta continuamente a los cambios.

Agile Unified Process es una versión simplificada y ágil del Proceso Unificado tradicional, manteniendo sus fortalezas pero eliminando su rigidez.

Crystal Clear es una metodología ligera diseñada específicamente para equipos pequeños, enfocándose en la comunicación y la simplicidad.

PMI Agile es el enfoque ágil desarrollado por el Project Management Institute, integrando prácticas ágiles con la gestión de proyectos tradicional.

Capítulo 3

Modelado del software

Índice general

3.1. Requisitos del software	18
3.1.1. Principios de comunicación	18
3.1.2. Definición de requisito	18
3.1.3. Tipos de Requisitos	18
Funcionales	18
No Funcionales	18
3.1.4. Clasificación de requisitos no funcionales	19
1. Product Requirements (Requisitos del Producto)	19
2. Process Requirements (Requisitos del Proceso)	19
3. External Requirements (Requisitos Externos)	20
3.1.5. Dificultades Comunes	20
Impacto	21
3.1.6. Ingeniería de requisitos	21
1. Indagación de requisitos	21
2. Elaboración	22
3. Negociación y especificación	24
4. Validación	24
5. Gestión	24
3.1.7. Modelado de requisitos	24
3.2. Diseño del software	25
3.2.1. Definición y propósito	25
3.2.2. Importancia del diseño	25
3.2.3. Atributos de calidad del software	25
3.2.4. Conceptos clave del diseño	25
3.2.5. Clases de diseño	26
3.2.6. Elementos del modelo de diseño	26
3.2.7. Diseño de arquitectura y patrones	26
MVC (Modelo-Vista-Controlador)	26
SOA (Arquitectura Orientada a Servicios)	27
EDA (Arquitectura Dirigida por Eventos)	27
Otros Patrones	27
3.2.8. Diseño de interfaz de usuario	28
Principio de familiaridad en el diseño de interfaces	28
Proceso de diseño de interfaz de usuario	28
3.2.9. Componentes	29

Identificación de componentes:	29
3.2.10. Revisión técnica formal	29

3.1. Requisitos del software

3.1.1. Principios de comunicación

1. Escuchar activamente.
2. Prepararse previamente.
3. Facilitar la comunicación (agenda, mediador).
4. Comunicación cara a cara.
5. Documentar decisiones clave.
6. Fomentar colaboración.
7. Seguir agenda.
8. Apoyar con representación gráfica.
9. Avanzar ante bloqueos.
10. Negociar de forma equilibrada.

3.1.2. Definición de requisito

Definición

Requisito: Descripción detallada de una funcionalidad o característica deseada. Guía el diseño, implementación y prueba.

3.1.3. Tipos de Requisitos

Funcionales

Qué: Describe qué debe hacer el software

Ejemplo

- Permitir ingresar información de contacto
- Calcular impuesto sobre renta
- Generar informe contable

No Funcionales

Cómo: Describe cómo debe comportarse el software

Ejemplo

- Disponibilidad 24/7
- Compatibilidad multi-SO/navegador
- Tiempo de respuesta ¡1 minuto

3.1.4. Clasificación de requisitos no funcionales

Los Requisitos No Funcionales se dividen en tres categorías principales:

1. Product Requirements (Requisitos del Producto)

Definen las características intrínsecas del software:

- Usability Requirements (Requisitos de Usabilidad)
 - Facilidad de uso e interfaz intuitiva
 - Experiencia del usuario (UX)
 - Accesibilidad
- Portability Requirements (Requisitos de Portabilidad)
 - Capacidad de ejecutarse en diferentes plataformas
 - Adaptabilidad a distintos sistemas operativos
 - Compatibilidad multiplataforma
- Efficiency Requirements (Requisitos de Eficiencia)
 - Uso optimizado de recursos del sistema
 - Gestión eficiente de memoria y CPU
 - Tiempos de respuesta
- Reliability Requirements (Requisitos de Confiabilidad)
 - Disponibilidad del sistema
 - Tolerancia a fallos
 - Recuperación ante errores
- Performance Requirements (Requisitos de Rendimiento) (subtipo de Eficiencia)
 - Velocidad de procesamiento
 - Throughput (rendimiento)
 - Tiempos de respuesta específicos
- Space Requirements (Requisitos de Espacio) (subtipo de Eficiencia)
 - Limitaciones de almacenamiento
 - Uso de memoria
 - Capacidad de datos

2. Process Requirements (Requisitos del Proceso)

Relacionados con el desarrollo y despliegue:

- Delivery Requirements (Requisitos de Entrega)
 - Plazos de desarrollo
 - Metodología de entrega
 - Hitos del proyecto

- Implementation Requirements (Requisitos de Implementación)
 - Lenguajes de programación específicos
 - Herramientas de desarrollo
 - Estándares de codificación
- Standards Requirements (Requisitos de Estándares)
 - Cumplimiento de normativas
 - Estándares de la industria
 - Protocolos específicos

3. External Requirements (Requisitos Externos)

Impuestos por factores externos al sistema:

- Interoperability Requirements (Requisitos de Interoperabilidad)
 - Integración con otros sistemas
 - Intercambio de datos
 - APIs y protocolos de comunicación
- Ethical Requirements (Requisitos Éticos)
 - Consideraciones morales
 - Impacto social
 - Responsabilidad corporativa
- Legislative Requirements (Requisitos Legislativos)
 - Cumplimiento legal
 - Regulaciones gubernamentales
 - Normativas específicas del sector
- Privacy Requirements (Requisitos de Privacidad) (subtipo de Legislativos)
 - Protección de datos personales
 - GDPR/LOPD
 - Gestión de información sensible
- Safety Requirements (Requisitos de Seguridad) (subtipo de Legislativos)
 - Protección contra amenazas
 - Seguridad de datos
 - Control de acceso

3.1.5. Dificultades Comunes

- Alcance: Fronteras mal definidas
- Entendimiento: Usuarios inseguros, información «obvia» omitida, requisitos ambiguos o en conflicto, ...

- Volatilidad: Requisitos cambian con el tiempo

Impacto

Estas dificultades pueden llevar a:

- 15 % del tiempo en definir requisitos.
- Cambiar requisitos tras entrega cuesta de 60 a 100 veces más.
- 56 % de errores por especificación incorrecta.
- Solo 2 % del software entregado cumple completamente.

3.1.6. Ingeniería de requisitos

Para desarrollar un software de calidad, es fundamental una buena ingeniería de requisitos. Para ello, se sigue un proceso iterativo que incluye:

- **Concepción** Necesidad inicial. Identificar la necesidad del software y su contexto.
- **Indagación** Preguntas organizadas sobre objetivos. ¿Qué, por qué y cómo?
- **Elaboración:** Modelo refinado, escenarios usuario, diagramas
- **Negociación:** Priorizar, resolver conflictos
- **Especificación:** Documentación formal
- **Validación:** Detectar ambigüedades, inconsistencias y verificar con usuarios
- **Administración:** Controlar cambios, versiones y trazabilidad

1. Indagación de requisitos

La indagación de requisitos es una fase crítica donde se obtienen y refinan los requisitos del software. En esta fase distinguen dos actividades principales:

1. Preparación

Antes de capturar los requisitos, es esencial una fase de preparación que garantice el éxito de la indagación. Esta comienza con la **identificación de los participantes**, es decir, todas las personas relevantes para el sistema: solicitantes, usuarios finales y otros perfiles implicados.

A continuación, se realiza la **recepción y análisis de la concepción**, un documento preliminar que describe de forma general la necesidad o el problema. Esta revisión ayuda a entender el contexto, anticipar posibles temas clave y preparar la discusión.

Una vez analizada la concepción, se elabora una **agenda de indagación** que guíe las reuniones y asegure que se aborden todos los puntos críticos:

- Quién es el solicitante.
- Qué perfil tienen los usuarios.
- Cuál es el problema a resolver.
- Cómo se definiría una solución exitosa.
- Qué otros participantes deben considerarse en el proceso.

2. Captura de requisitos

Durante la indagación, los requisitos pueden obtenerse mediante:

- Entrevistas estructuradas.
- Lluvia de ideas (brainstorming).
- Observación directa del entorno.
- Análisis de sistemas similares.
- Prototipos de baja fidelidad.

3. Tipos de requisitos según el cliente

- **Normales:** Son los que el cliente solicita explícitamente.
- **Esperados:** No se mencionan, pero se asumen necesarios.
- **Emocionantes:** Superan las expectativas del cliente y generan satisfacción.

2. Elaboración

La elaboración es una fase donde se refinan los requisitos obtenidos en la indagación.

Casos de uso

Definición

Un **caso de uso** describe una secuencia de acciones entre un actor y el sistema. Representa una unidad de funcionalidad y permite estimar, verificar y priorizar el trabajo. Cada caso puede dividirse en varios escenarios (principales o alternativos), y cada actor representa un conjunto de roles del usuario.

Características

- Valorados por clientes
- Estimables (implementación independiente)
- Pequeños (fácil estimación)
- Verificables

Aspecto	Casos Uso	Historias Usuario
Agilismo	Indiferente	Sí
Representan requisitos	Sí	No
Describen	Interacciones actor-sistema	Capacidades usuario
Útil para	Stakeholders/Clientes	Equipo desarrollo

Cuadro 3.1: Comparación entre Casos de Uso y Historias de Usuario

Ejemplo**Especificación de un caso de uso:**

- **Nombre:** Nombre del caso de uso
- **Actor/es:** Rol del usuario (o entidad(es)) que interactúa con el sistema
- **Descripción:** Breve resumen de la funcionalidad
- **Precondiciones:** Condiciones que deben cumplirse antes de iniciar el caso de uso, se asumen ciertas.
- **Dependencias:** Otros casos de uso o requisitos que deben cumplirse
- **Escenario principal:** Secuencia de pasos que describe la interacción normal entre el actor y el sistema
- **Excepciones:** Condiciones especiales o errores que pueden ocurrir durante la ejecución del caso de uso
- **Prioridad (P0, P1, ...):** Importancia del caso de uso

Historias de usuario**Definición**

Descripción breve y sencilla, escrita en lenguaje natural y enfocada en las necesidades del usuario, que sirve como base para el desarrollo de una característica del software.

Ejemplo

«[rol], quiero [función] para [beneficio]»

Una historia específica podría ser:

Ejemplo

Como usuario de la aplicación de compras quiero ver el estado de mis pedidos para saber cuáles están en proceso de envío.

No son requisitos ni sustituyen a los casos de uso, pero ayudan a centrar el desarrollo en el valor para el usuario y permiten definir criterios de aceptación. En la Tabla 3.1 se muestra una comparación entre casos de uso y historias de usuario.

Casos de uso vs Historias de usuario

- **Casos de uso:** Más detallados, útiles para stakeholders.
- **Historias de usuario:** Más ágiles, enfocadas al equipo de desarrollo.

Ambos permiten priorización y ponen al usuario en el centro.

3. Negociación y especificación

Durante la negociación, se ajustan expectativas, prioridades y limitaciones. Se analiza el coste, el tiempo y el alcance del producto. De este proceso resultan dos documentos clave:

- **DRU (Documento de Requisitos del Usuario):** Qué problema resolver.
- **ERS (Especificación de Requisitos del Software):** Qué debe hacer el sistema para alcanzar los objetivos del DRU.

4. Validación

Se realiza para asegurar:

- Coherencia entre requisitos y resultado esperado.
- Priorización adecuada.
- Ausencia de ambigüedades.
- Viabilidad y resolución de conflictos.

Técnicas: revisión por expertos, prototipado, validación de modelos, verificación de testabilidad.

5. Gestión

La gestión consiste en supervisar los requisitos a lo largo de todo el ciclo de vida del producto. Incluye:

- Registro de cambios y asegurar la consistencia entre los requisitos y el sistema.
- Repriorización.
- Adición o eliminación de requisitos.
- Seguimiento del estado de cada uno.

3.1.7. Modelado de requisitos

El modelado de requisitos tiene como objetivo representar, de forma gráfica o estructurada, los distintos aspectos del sistema desde el punto de vista de los usuarios, del dominio de información y del comportamiento del software. Permite comunicar mejor las necesidades del sistema, detectar inconsistencias y preparar el diseño.

Tipos de diagramas:

- **Basados en escenarios:** describen interacciones usuario-sistema. Incluyen casos de uso y sus escenarios.
- **De datos:** muestran entidades, atributos y relaciones (diagramas E-R).
- **Orientados a clases:** presentan clases, atributos, métodos y relaciones (herencia, asociación. . .).
- **De flujo:** representan transformación y circulación de datos.
- **De comportamiento:** representan estados del sistema y transiciones ante eventos.

Lenguaje de modelado UML:

- **Diagramas estructurales:** clases, objetos, componentes, despliegue.
- **Diagramas de comportamiento:** casos de uso, secuencia, estados, colaboración.

Ventajas del modelado:

- Mejora la comprensión y comunicación.
- Detecta errores e inconsistencias tempranas.
- Sirve como base para diseño y validación.

3.2. Diseño del software

3.2.1. Definición y propósito

Definición

El diseño de software consiste en crear un modelo que describa la arquitectura del sistema, los componentes que lo conforman, sus interfaces y las relaciones entre ellos. Se busca traducir los requisitos en una estructura organizada que guíe a la implementación.

Propósitos principales:

- Facilitar la comunicación entre los participantes del proyecto.
- Servir de base para el análisis y la toma de decisiones.
- Identificar y gestionar las abstracciones clave del sistema.
- Promover la reutilización a gran escala (familias de productos).

3.2.2. Importancia del diseño

Un buen diseño:

- Permite traducir correctamente los requisitos en soluciones funcionales.
- Reduce el riesgo de errores, inestabilidad y dificultad de pruebas.
- Sienta las bases de la calidad del software.

3.2.3. Atributos de calidad del software

- **Funcionalidad:** capacidades del sistema, cobertura y seguridad.
- **Usabilidad:** facilidad de uso, estética, documentación.
- **Confiabilidad:** frecuencia y gravedad de fallos (MTBF, MTTR).
- **Rendimiento:** velocidad, tiempo de respuesta, uso de recursos.
- **Mantenibilidad:** facilidad para modificar, adaptar y probar.

3.2.4. Conceptos clave del diseño

- **Abstracción:** ocultar detalles para centrarse en lo esencial.
- **Arquitectura:** organización general del sistema y sus relaciones.
- **Patrones de diseño:** soluciones reutilizables a problemas comunes.
- **División de problemas:** estrategia de «divide y vencerás».
- **Modularidad:** división del software en componentes independientes.

- **Ocultamiento de información:** cada módulo expone solo lo necesario.
- **Independencia funcional:** cada módulo resuelve tareas bien definidas.
- **Cohesión y acoplamiento:** coherencia interna y dependencia entre módulos.
- **Rediseño:** mejora estructural sin alterar funcionalidad.

3.2.5. Clases de diseño

- **Interfaz de usuario:** elementos visibles e interactivos.
- **Dominio de negocio:** lógica y entidades principales.
- **Procesos:** gestión de tareas o flujos. Abstracciones de alto nivel que definen cómo se ejecutan las tareas.
- **Persistencia:** almacenamiento y recuperación de datos.
- **Sistema:** gestión de la infraestructura software.

3.2.6. Elementos del modelo de diseño

- **Diseño de datos:** organización de estructuras y bases de datos.
- **Diseño arquitectónico:** definición de subsistemas.
- **Diseño de interfaces:** entre componentes, usuarios y otros sistemas.
- **Diseño de componentes:** estructuras internas, algoritmos, interfaces.
- **Diseño de despliegue:** mapeo lógico-físico (servidores, contenedores).

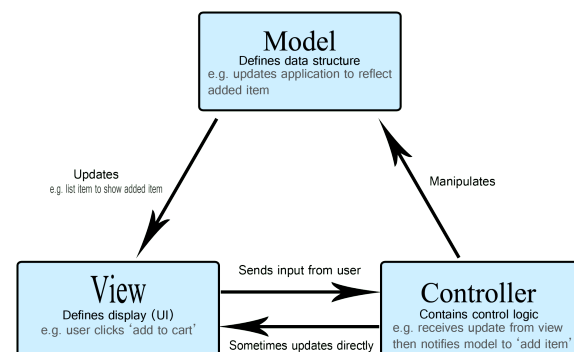
3.2.7. Diseño de arquitectura y patrones

Definición

Arquitectura: Estructura o estructuras del sistema, lo que comprende a los componentes del software, sus propiedades externas visibles y las relaciones entre ellos.

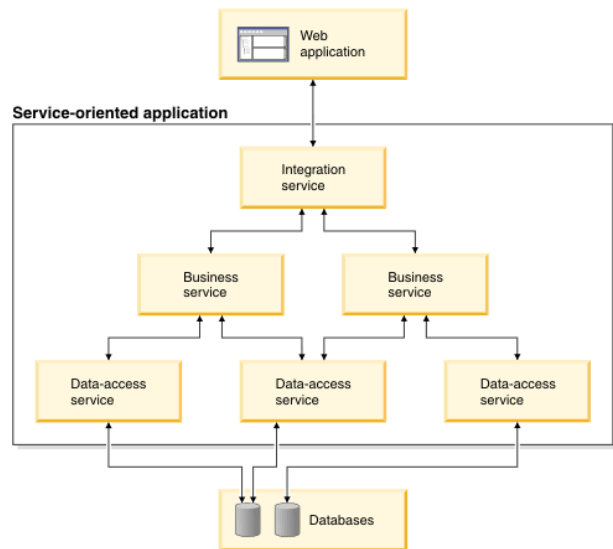
MVC (Modelo-Vista-Controlador)

- **Modelo:** Datos y lógica negocio
- **Vista:** Interfaz usuario (presentación)
- **Controlador:** Mediador (recibe inputs, actualiza modelo)
- **Flujo:**
 1. Usuario → Acción → Controlador
 2. Controlador → Actualiza Modelo
 3. Modelo → Notifica → Vista
 4. Vista → Renderiza → Usuario
- **Ventajas:** Separación preocupaciones, escalable
- **Uso:** Aplicaciones con UI compleja



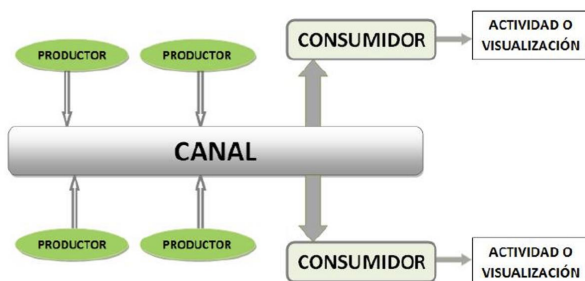
SOA (Arquitectura Orientada a Servicios)

- Componentes autónomos (**Servicios**)
- Comunicación mediante APIs
- Niveles:
 - Integración
 - Lógica negocio
 - Acceso datos
- **Ventajas:** Bajo acoplamiento, escalabilidad horizontal
- **Desafío:** Complejidad integración



EDA (Arquitectura Dirigida por Eventos)

- Componentes: **Productores** (generan eventos) y **Consumidores** (reaccionan)
- Comunicación asíncrona mediante eventos
- **Ventajas:** Escalabilidad, tiempo real
- **Uso:** Sistemas IoT, alta concurrencia



Otros Patrones

- **Layered:** Organización en capas (presentación, negocio, datos)
- **Microservicios:** SOA con servicios más pequeños y especializados

- **Workflow:** Orquestación de procesos secuenciales

3.2.8. Diseño de interfaz de usuario

Definición

El diseño de interfaz de usuario (UI) es el proceso de crear componentes visuales y funcionales que permiten a los usuarios interactuar con el software. Busca optimizar la experiencia del usuario (UX) y facilitar la usabilidad. Recoge los datos del subsistema de lógica de negocio para mostrarla. Reenvía las interacciones del usuario al subsistema de lógica de negocio para su procesamiento.

Tipos: GUI (gráfica), TUI (texto), VUI (voz).

Aspectos clave:

- Usabilidad (eficiencia, satisfacción, accesibilidad).
- Personas: perfiles de usuario y escenarios de uso.
- Aspectos: claridad, tiempo de respuesta, ayuda, errores, internacionalización.
- Principios: familiaridad, uniformidad, recuperabilidad, diversidad.

Principio de familiaridad en el diseño de interfaces

El principio de familiaridad busca que el usuario se sienta cómodo desde el primer uso de la interfaz, aprovechando convenciones conocidas o comportamientos esperados. Esto facilita la adopción del sistema, mejora la usabilidad y reduce la curva de aprendizaje.

Algunas técnicas comunes asociadas a este principio son:

- **Lazy registration:** permite que los usuarios accedan parcialmente a la funcionalidad sin necesidad de autenticarse inicialmente. Solo cuando la acción lo requiere, se solicita el registro o inicio de sesión.
 - **Propósito:** evitar fricción en el primer uso y fomentar la adopción.
- **Breadcrumbs (migas de pan):** muestran la ruta de navegación desde la página principal hasta la ubicación actual.
 - **Propósito:** permitir al usuario orientarse y volver atrás con facilidad.
- **Hover control:** oculta información secundaria y solo la muestra cuando el usuario sitúa el cursor encima del elemento.
 - **Propósito:** reducir la carga cognitiva y evitar distracciones.
- **Search filters:** ofrece comandos o filtros de búsqueda predeterminados o sugerencias.
 - **Propósito:** facilitar la exploración y ayudar al usuario a reconocer lo relevante.
- **Carrito de la compra (e-commerce):** el carrito es accesible desde cualquier sección del sitio.
 - **Propósito:** mejorar la experiencia de compra y aumentar la conversión.

Proceso de diseño de interfaz de usuario

1. Comprensión de actividades del usuario.

2. Arquitectura de información.
3. Prototipado de baja fidelidad.
4. Pruebas de usabilidad.
5. Diseño final (alta fidelidad, posible código).

3.2.9. Componentes

Definición

Componentes: unidades reutilizables del sistema que encapsulan funcionalidad. Cada componente tiene una interfaz bien definida y puede interactuar con otros componentes. Los componentes pueden ser independientes o formar parte de un subsistema mayor.

Identificación de componentes:

La correcta identificación de componentes es fundamental para lograr un diseño robusto, modular y mantenible. Se deben tener en cuenta los siguientes criterios:

- **Relación directa con los requisitos:** cada componente debe corresponderse con una funcionalidad significativa descrita en los requisitos del sistema.
- **Agrupación lógica:** las responsabilidades del componente deben estar relacionadas funcional o temáticamente.
- **Fronteras bien definidas:** cada componente debe tener una interfaz clara y estable que delimite lo que ofrece al resto del sistema.
- **Bajo acoplamiento:** los componentes deben minimizar sus dependencias mutuas, de modo que un cambio en uno no afecte a los demás.
- **Alta cohesión:** todas las funciones internas del componente deben estar fuertemente relacionadas y contribuir a un único propósito.
- **Reutilización potencial:** debe evaluarse si el componente puede ser útil en otros proyectos o contextos.
- **Autonomía en el despliegue:** en arquitecturas modernas (como microservicios), es deseable que cada componente pueda desplegarse y escalarse de forma independiente.

Ejemplos de componentes:

- Módulo de autenticación y gestión de usuarios.
- Servicio de generación de informes PDF.
- API de gestión de pedidos.
- Componente de acceso a base de datos (DAO).
- Microservicio de pagos.

3.2.10. Revisión técnica formal

Objetivo: detectar problemas antes de la implementación.

Participantes: equipo de desarrollo.

Evaluación de: mantenibilidad, seguridad, disponibilidad, rendimiento, funcionalidad, usabilidad, coste.

Capítulo 4

Planificación de proyectos

Índice general

4.1. Estimación	32
4.1.1. Objetivo y proceso de la planificación	32
4.1.2. Ámbito vs Factibilidad	32
4.1.3. Técnicas de Estimación	33
Ley de Parkinson	33
Precio Oportunista	33
4.1.4. Técnicas de Descomposición	33
Tipos de Descomposición	33
Estimación Basada en Problema	33
Estimación Basada en Proceso	34
Modelos Empíricos	34
4.1.5. COCOMO II	34
Conceptos Básicos	34
Tipos de Proyectos	35
Ejemplo Práctico	35
Limitaciones	35
4.1.6. Estimación Ágil	35
Poker Planning	35
4.2. Calendarización	35
4.2.1. Relación con la Estimación	35
4.2.2. Dificultades Comunes	36
4.2.3. Manejo de Fechas Límite	36
4.2.4. Hitos y Entregas	36
4.2.5. Dependencias entre Tareas	37
4.2.6. Diagrama de Gantt	37
4.2.7. Diagrama de PERT	37
Conceptos fundamentales	37
Elementos	38
Beneficios	38
4.3. Administración del Riesgo	38
4.3.1. Clasificación de Riesgos	39
4.3.2. Descripción de Riesgos	39
4.3.3. Estrategias de Gestión de Riesgos	40
4.3.4. Proceso de Gestión de Riesgos	41
4.3.5. Actividades por fase	41

4.1. Estimación

4.1.1. Objetivo y proceso de la planificación

La planificación busca determinar los recursos necesarios para completar el proyecto en plazo y con calidad aceptable. Los elementos principales son:

- Estimación del esfuerzo y tiempo
- Asignación de tareas
- Identificar las dependencias
- Gestión de riesgos

Sus fases son:

1. **Establecer ámbito:** ¿Qué se va a hacer? (casos de uso + requisitos no funcionales)
2. **Determinar viabilidad:** ¿Es posible? (tecnología, finanzas, tiempo)
3. **Analizar riesgos:** ¿Qué puede salir mal?
4. **Definir recursos:** Personal, hardware, herramientas y componentes reutilizables
5. **Estimar coste/esfuerzo:** Técnicas como COCOMO o puntos de historia
6. **Desarrollar calendario:** Con tareas, hitos y dependencias

4.1.2. Ámbito vs Factibilidad

El ámbito puede ser tanto una descripción escrita como un conjunto de casos de uso. Se deben tener en cuenta los detalles de los casos de uso y considerar las restricciones.

Por otro lado, la factibilidad es hasta qué punto es realizable el proyecto. Esta puede ser:

- Tecnológica
- Financiera
- Temporal
- Material (recursos)

Los recursos se pueden dividir de la siguiente forma dentro del proyecto:

- **Personal:**
 - Número
 - Habilidades
 - Ubicación
- **Entorno:**
 - Herramientas software
 - Hardware
 - Recursos de red
- **Software reutilizable:**
 - Componentes nuevos

- **Componentes de experiencia parcial:** Las especificaciones, los diseños, código o pruebas existentes de proyectos anteriores podrán ser usados para el proyecto actual, pero requerirán una modificación sustancial y los miembros del equipo han limitado su experiencia sólo al área de aplicación representada por los componentes. Por eso, las modificaciones tendrán un mayor riesgo
- **Componentes con experiencia completa:** Ya existentes y desarrollados para proyectos anteriores similares al software que se va a construir para el proyecto actual. Los miembros del equipo ya tienen experiencia en el área. Bajo riesgo
- **Componentes COTS:** Diseñados para un uso inmediato, no requiere modificaciones. Sin riesgo

4.1.3. Técnicas de Estimación

Ley de Parkinson

El mismo trabajo requiere más tiempo cuando hay plazos más largos.

Precio Oportunista

- Basado en lo que el cliente está dispuesto a pagar
- Común en concursos públicos y licitaciones
- Riesgo: Puede no reflejar el esfuerzo real requerido

4.1.4. Técnicas de Descomposición

Tipos de Descomposición

- **Lógica difusa:**
 - Identificar tipo de aplicación
 - Establecer magnitud en escala cualitativa
 - Refinar dentro del rango original
- **Puntos de Función (PF):**
 - Medir características del dominio de información
 - Componentes: Entradas, Salidas, Consultas, Archivos, Interfaces
- **Componente Estándar:**
 - Contar ocurrencias de componentes estándar
 - Usar datos históricos para estimar tamaño por componente
- **Dimensionamiento del Cambio:**
 - Para proyectos que modifican software existente
 - Estimar número y tipo de modificaciones requeridas

Estimación Basada en Problema

- **Métricas clave:**
 - Líneas de Código (LOC)

- Puntos de Función (PF)

■ **Fórmula de estimación ponderada:**

$$S = \frac{S_{\text{opt}} + 4S_m + S_{\text{pes}}}{6}$$

donde:

- S_{opt} = Estimación optimista
- S_m = Estimación más probable
- S_{pes} = Estimación pesimista

■ **Ejemplo práctico:**

Componente	Optimista	Probable	Pesimista	Estimación
GUI	4600	6900	8600	6800
Servicio 1	2200	2750	3300	2750
Servicio 2	2500	3300	4400	3350
Servicio 3	1800	2250	2700	2250
Total	15150 LOC			

■ **Cálculo de coste:**

$$\text{Esfuerzo} = \frac{\text{Total LOC}}{\text{Productividad}} = \frac{15150}{750} = 20,2 \text{ persona-meses}$$

Estimación Basada en Proceso

- Descomposición en actividades del proceso de software
- Asignación de esfuerzo a cada actividad

Modelos Empíricos

■ **Fórmula general:**

$$E = A + B \cdot (e_v)^C$$

- E : Esfuerzo en personas-mes.
- e_v : Variable de estimación (LOC o PF).
- A, B, C : Constantes derivadas de datos históricos.

- **Ventaja:** Predictibilidad mediante regresión sobre proyectos pasados.

4.1.5. COCOMO II

Conceptos Básicos

- **Objetivo:** Estimar esfuerzo (E) y tiempo (D) en función del tamaño (KLOC) para (N) personas
- **Ecuaciones:**

$$E = a \cdot (\text{KLOC})^b$$

$$D = c \cdot (E)^d$$

Tipos de Proyectos

Tipo de Proyecto	a	b	c	d
Orgánico (pequeño, requisitos flexibles)	2,4	1,05	2,5	0,38
Semi-acoplado (complejidad media)	3,0	1,12	2,5	0,35
Empotrado (requisitos rígidos)	3,6	1,20	2,5	0,32

Ejemplo Práctico

- **Tamaño total:** 15.150 LOC = 15,15 KLOC
- **Proyecto orgánico:**

$$E = 2,4 \cdot (15,15)^{1,05} = 42 \text{ personas-mes}$$

$$D = 2,5 \cdot (42)^{0,38} = 11 \text{ meses}$$

$$N = \frac{E}{D} = \frac{42}{11} = 4 \text{ personas}$$

Limitaciones

- No considera reusabilidad en programación orientada a objetos
- Basado en muestras limitadas (no aplicable a todos los entornos)
- Ignora paralelización de tareas y factores de productividad

4.1.6. Estimación Ágil

Poker Planning

Paso 1: Seleccionar una historia de usuario

Paso 2: Discusión breve del equipo

Paso 3: Estimación individual con tarjetas (ejemplo: Fibonacci: 1, 2, 3, 5, 8)

Paso 4: Revelar estimaciones simultáneamente

Paso 5: Si hay discrepancia, debatir y repetir

Puntos vs. Horas

Puntos		Horas	
Ventajas	▪ Capturan complejidad, riesgo y esfuerzo ▪ Enfocados en valor (no tiempo)	Ventajas	▪ Fácil medición del trabajo ▪ Cálculo directo de productividad
Desventajas	▪ Abstractos (requieren equipo consolidado)	Desventajas	▪ Ignoran trabajo no relacionado (reuniones, etc.)

4.2. Calendarización

4.2.1. Relación con la Estimación

Estimación vs Calendarización:

Aspecto	Estimación	Calendarización
Propósito	Calcular esfuerzo/tiempo	Fijar fechas y paralelización
Paralelismo	No considera tareas simultáneas	Gestiona dependencias
Entregas	Sin fechas específicas	Establece hitos y entregas
Ejemplo:	“Requiere 40 personas-mes”	“Versión 1.0 para 15/Oct”

4.2.2. Dificultades Comunes

1. Fechas límite **irreales** impuestas externamente
2. **Cambios constantes** en requisitos del cliente
3. **Subestimación** de esfuerzo/recursos
4. **Riesgos no considerados** inicialmente
5. Problemas técnicos **imprevistos**
6. **Falta de comunicación** en el equipo
7. Negación de **retrasos** y correcciones

4.2.3. Manejo de Fechas Límite

Estrategias efectivas:

- **Estimación detallada:** Usar datos históricos de proyectos similares
- **Desarrollo incremental:**
 - Establecer qué funcionalidad corresponde a cada entrega
 - Asegurar las funcionalidades mínimas básicas antes de cada entrega
- **Comunicación proactiva:**
 1. Demostrar con datos por qué la fecha es irreal
 2. Proponer alternativas concretas
 3. Negociar alcance vs. tiempo
- **Enfoque ágil:**
 - **Sprints** cortos con entregables concretos
 - **Priorización flexible** con el cliente

4.2.4. Hitos y Entregas

- **Hito válido:**
 - Final de etapa lógica (ejemplo: «Prototipo móvil terminado»)
 - Medible y verificable
 - Responsable asignado
- **Anti-ejemplo:**
 - «50 % del código funciona» (no verificable)
- **Entrega:**

- Hito entregado al cliente
- Ejemplo: «Versión con registro de usuarios»
- Debe incluir criterios de aceptación claros

4.2.5. Dependencias entre Tareas

Tipo	Notación	Descripción
Fin-Inicio	FS	B no empieza hasta que A termina (<i>ejemplo: Pruebas después de desarrollo</i>)
Inicio-Inicio	SS	B no empieza hasta que A empieza (<i>ejemplo: Diseño UI y «back-end»</i>)
Fin-Fin	FF	B no termina hasta que A termina (<i>ejemplo: Documentación y codificación</i>)
Inicio-Fin	SF	B no termina hasta que A empieza (<i>rara en software</i>)

Ejemplos prácticos:

- **FS:** Pruebas no pueden comenzar hasta que desarrollo termine
- **SS:** Diseño UI y diseño DB pueden comenzar simultáneamente
- **FF:** Documentación no puede completarse hasta que desarrollo finalice
- **SF:** Mantenimiento no puede terminar hasta que despliegue comience

4.2.6. Diagrama de Gantt

Se muestra el diagrama de Gantt en la Figura 4.1.

- **Elementos clave:**
 - Tareas en eje vertical
 - Tiempo en eje horizontal
 - Barras horizontales = duración
- **Ventajas:**
 - Visualización clara del cronograma
 - Identificación de solapamientos
 - Seguimiento de progreso
- **Ejemplo:**

4.2.7. Diagrama de PERT

Proporciona una representación visual del cronograma de un proyecto y desglosa las tareas individuales.

Conceptos fundamentales

- **Nodos:** Representan eventos (no actividades)
- **Flechas:** Actividades con duración
- **Camino crítico:** Ruta más larga (determina duración total)

Actividad	Octubre		Noviembre							
	28	30	1	2	4	6	8	9	10	11
Escoger tema de trabajo										
Definir objetivo general										
Definir objetivo específico										
Asignación de roles										
Diseño y desarrollo del código										
Diseño de la wiki										
Desarrollo de la wiki										
Consultas externas o asesorías										
Últimas modificaciones										
Entrega										

Figura 4.1: Diagrama de Gantt

Elementos

Este diagrama tiene los elementos de la Tabla 4.1.

Término	Significado
D (Duración)	Tiempo requerido para realizar la actividad
ES (Early Start)	Primer momento posible de inicio
EF (Early Finish)	Primer momento posible de fin
LS (Late Start)	Último momento posible de inicio
LF (Late Finish)	Último momento posible de fin
TF (Holgura Total)	Máximo retraso sin afectar proyecto
FF (Holgura Libre)	Máximo retraso sin afectar al «inicio temprano» de «sucesoras» inmediatas
Camino Crítico	Secuencia de tareas más larga. Determinará la duración mínima del proyecto

Cuadro 4.1: Elementos del diagrama PERT

Beneficios

- Identifica cuellos de botella
- Calcula probabilidades de cumplimiento
- Gestiona riesgos de tiempo

4.3. Administración del Riesgo

- **RAE:** «Contingencia o proximidad de un daño»
- **Ingeniería del software:**
 - Posibilidad de eventos imprevistos con impacto negativo en el proyecto
 - Implica previsión del futuro, cambio y toma de decisiones
- **Elementos clave:**
 - **Fuente:** Origen del riesgo (ejemplo: nueva tecnología)
 - **Evento:** Qué podría ocurrir (ejemplo: incompatibilidad)

- **Impacto:** Consecuencias (ejemplo: retraso 2 semanas)

4.3.1. Clasificación de Riesgos

Tipo de Riesgo	Descripción y Ejemplos
Proyecto	▪ Calendario o recursos insuficientes ▪ Ejemplo: Desarrollo por detrás de las metas acordadas con el cliente
Técnico	▪ Amenaza calidad o planificación ▪ ejemplo: Rendimiento inadecuado bajo carga máxima
Producto	▪ Incumplimiento expectativas usuario ▪ ejemplo: Interfaz poco intuitiva para usuarios finales
Negocio	▪ Problemas organizativos o de mercado ▪ ejemplo: Cambio en estrategia corporativa que reduce la prioridad del proyecto
Tipo de Riesgo	Descripción y Ejemplos
Tecnologías	▪ Librerías obsoletas sin soporte ▪ Dependencias con licencias restrictivas
Personal	▪ Baja capacitación en tecnología clave ▪ Conflictos interpersonales en equipo
Organizativos	▪ Cambios frecuentes en liderazgo ▪ Presupuesto inestable
Herramientas	▪ Incompatibilidad entre entornos ▪ Bugs en software de testing
Requisitos	▪ Alcance no definido (scope creep) ▪ Requerimientos contradictorios
Estimación	▪ Subestimación de complejidad ▪ Falta de datos históricos

4.3.2. Descripción de Riesgos

Para cada riesgo identificado, se recomienda recopilar la siguiente información:

- **Descripción:**Cuál es el riesgo y cuál es su origen.
- **Prioridad (impacto):** *alta — media — baja*
- **Probabilidad:** *alta — media — baja*
- **Plan de acción:**
 - Tipo: *contingencia, minimización o prevención*

- Estrategia: *soportar, reducir el impacto, evitar*
- Acciones planificadas: descripción específica de los pasos a seguir.
- **Responsable:** Persona a cargo de la implementación del plan
- **Estado:** *abierto — cerrado*

Ejemplo

Descripción: Posible ausencia prolongada de una persona clave del equipo.

Impacto: Alto

Probabilidad: Promedio

Plan de contingencia:

- Tipo: Contingencia
- Estrategia: Reducir el impacto
- Acciones: Documentar el trabajo clave y la capacitación cruzada dentro del equipo

Responsable: Coordinador del proyecto

Estado: Abierto

4.3.3. Estrategias de Gestión de Riesgos

La gestión de riesgos puede adoptar diferentes enfoques según el momento oportuno para actuar sobre el riesgo:

- **Estrategias reactivas:** Se actúa tras la materialización del riesgo, evaluando sus consecuencias y tomando medidas correctivas
- **Estrategias proactivas:** El riesgo se considera un posible evento futuro. Se establece un plan de contingencia para evitarlo o, en caso de ocurrir, minimizar su impacto

Un análisis temprano, sistemático y profundo de los riesgos favorece una estrategia proactiva global y reduce la ocurrencia de problemas imprevistos graves.

La gestión de riesgos es el proceso continuo que permite:

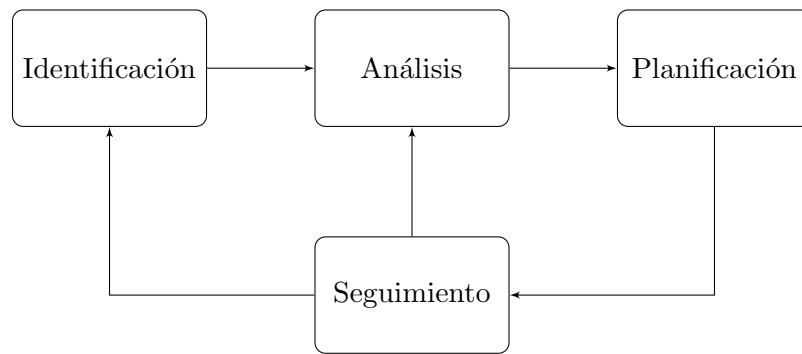
- Identificar riesgos potenciales
- Analizar su naturaleza e impacto
- Proponer soluciones antes de que se conviertan en problemas reales

Este proceso debe aplicarse continuamente durante todo el ciclo de vida del proyecto.

Nota personal

La integración temprana de la gestión de riesgos en la planificación permite anticipar fallos, tomar decisiones más informadas y reducir costos inesperados.

4.3.4. Proceso de Gestión de Riesgos



4.3.5. Actividades por fase

- **Identificación:** Listar riesgos potenciales
- **Análisis:** Calcular probabilidad/impacto, priorizar
- **Planificación:** Definir estrategias y planes de acción
- **Seguimiento:**
 - Revisión mensual de riesgos activos
 - Actualizar matriz de riesgos
 - Activar planes de contingencia si es necesario

Capítulo 5

Calidad

Índice general

5.1. Aseguramiento de la Calidad	43
5.1.1. Principios rectores de la calidad	43
5.1.2. Atributos de las métricas	43
5.1.3. Control vs. Aseguramiento de la Calidad	44
5.1.4. Revisiones técnicas	44
5.1.5. Revisiones durante el desarrollo	44
5.1.6. Métricas de calidad (DORA Metrics)	44
5.1.7. Buenas prácticas de desarrollo	45
5.1.8. Métricas de desarrollo	45
5.1.9. Deuda técnica	45
5.2. Estrategias de Prueba	46
5.2.1. Verificación vs. Validación	46
5.2.2. Malas prácticas	46
5.2.3. Estrategias de prueba (pirámide de pruebas)	46
5.2.4. Dimensiones de la prueba	46
5.2.5. Pruebas específicas	46
5.3. Administración de la Configuración	47
5.3.1. Ítems de Configuración (IC)	47
5.3.2. Sistema de Administración	47
5.3.3. Repositorio ACS	48
5.3.4. Proceso de Administración de la Configuración	48
5.3.5. Administración del contenido	48
5.3.6. Conflictos dominantes	48

Definición

La calidad en software se define como un proceso eficaz que, al ser bien aplicado, crea un producto útil con valor medible tanto para los productores como para los usuarios.

La ingeniería de software tiene como objetivo garantizar esa calidad a lo largo de todo el ciclo de vida:

- Planificación
- Diseño
- Desarrollo

- Pruebas
- Despliegue
- Mantenimiento

5.1. Aseguramiento de la Calidad

Definición

Según la norma ISO/IEC 9126, la calidad de un producto software se puede evaluar mediante seis atributos principales:

1. **Funcionalidad:** Grado en que el software cumple los requisitos funcionales esperados.
2. **Confiabilidad:** Estabilidad del software ante fallos, por ejemplo, el tiempo medio de funcionamiento antes de un fallo.
3. **Usabilidad:** Facilidad de uso para los usuarios.
4. **Eficiencia:** Uso óptimo de los recursos disponibles (CPU, memoria, etc.).
5. **Mantenibilidad:** Facilidad para modificar, corregir o mejorar el software.
6. **Portabilidad:** Facilidad para trasladar el software entre diferentes entornos.

Nota personal

Existe un dilema clásico entre producir software rápido y barato o producir software de alta calidad, ya que la calidad requiere tiempo y recursos.

Nota personal

Los errores pequeños no detectados a tiempo pueden amplificarse y causar problemas mayores en fases posteriores, por lo que la detección temprana es fundamental.

5.1.1. Principios rectores de la calidad

- **Formulación.** La derivación de medidas y métricas de software apropiadas
 - para la representación del software que se está construyendo.
- **Recolección.** Mecanismo que se usa para acumular datos requeridos para derivar las métricas formuladas.
- **Análisis.** El cálculo de métricas y la aplicación de herramientas
 - matemáticas.
- **Interpretación.** Evaluación de las métricas resultantes para comprender la calidad de la representación.
- **Retroalimentación.** Recomendaciones derivadas de la interpretación de las métricas del producto, transmitidas al equipo de software.

5.1.2. Atributos de las métricas

1. **Medible.** Debe ser simple poder recolectar los datos que componen la métrica y realizar su cálculo.

2. **Intuitiva.** Los usuarios de la métrica deben poder identificar su significado y su valor.
3. **Objetiva.** Siempre debe producir resultados que no tengan ambigüedades.
4. **Coherente.** El cálculo matemático de la métrica debe usar medidas que no conduzcan a combinaciones extrañas de unidades.
5. **Tecnológicamente agnóstica.** Debe basarse en el modelo de requerimientos, el modelo de diseño o la estructura del programa en sí.
6. **Accionable.** Debe proporcionar información que pueda conducir a un producto final de mayor calidad.

5.1.3. Control vs. Aseguramiento de la Calidad

Control de Calidad (QC)	Aseguramiento de Calidad (QA)
Reactivo	Proactivo
Detección y corrección de errores después de que ocurren	Prevención de errores mediante estándares y procesos
Inspección de productos	Mejora continua de procesos

5.1.4. Revisiones técnicas

- **Informales:** Conversaciones espontáneas, revisiones en escritorio; baja eficacia que mejora con listas de verificación.
- **Formales:** Reuniones estructuradas y preparadas; alta eficacia, más tiene un alto coste en tiempo y esfuerzo; se utiliza normalmente una muestra representativa.

5.1.5. Revisiones durante el desarrollo

- **Revisión de código:** Entre compañeros (pair review) para detectar errores y mejorar el aprendizaje del equipo.
- **Análisis estático de código:** Herramientas automáticas que detectan errores potenciales, complejidad y duplicaciones.

5.1.6. Métricas de calidad (DORA Metrics)

Definición

D.O.R.A.: DevOps Research and Assessment, un conjunto de métricas para evaluar la calidad del software y la eficiencia del equipo de desarrollo.

Métrica	Significado
MTTR (Mean Time to Recover)	Tiempo medio para recuperar un sistema tras un fallo
MTBF (Mean Time Between Failures)	Tiempo medio entre fallos
Disponibilidad	Proporción de tiempo que el sistema está disponible

Definición

La disponibilidad se calcula con la fórmula:

$$\text{Disponibilidad} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} \cdot 100 \%$$

5.1.7. Buenas prácticas de desarrollo

- **Clean Code** (Robert C. Martin - Uncle Bob):
 - KISS: “Keep It Simple, Stupid”
 - DRY: “Don’t Repeat Yourself”
 - YAGNI: “You Aren’t Gonna Need It”
 - SoC: “Separation of Concerns”
- **Documentación:** Comentarios explicativos (no descriptivos), explicaciones en los commits y ejemplos de uso en tests.
- **Control de versiones:** Uso de herramientas como Git para seguir cambios y facilitar la colaboración.

5.1.8. Métricas de desarrollo

Métrica	Descripción
Densidad de comentarios	% de comentarios respecto al código total
Duplicidad de código	Código repetido
Cobertura de pruebas	% de código ejecutado durante pruebas
Complejidad ciclomática	Mide rutas lógicas (condiciones y bucles)
IMS (Índice de Madurez del código(Software))	Evalúa la estabilidad de una release

Definición

La fórmula para calcular el IMS es:

$$IMS = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

Donde:

- M_T : Número total de pruebas planificadas.
- F_a : Fallos críticos encontrados.
- F_c : Fallos menores encontrados.
- F_d : Fallos detectados en desarrollo.

5.1.9. Deuda técnica

Definición

Según Martin Fowler, la deuda técnica es una metáfora financiera: tomar atajos en el diseño genera un «interés» que se paga con mayor esfuerzo futuro. Se puede:

- Seguir pagando intereses (mantener mal diseño).
- Pagar el principal (refactorizar y mejorar).

Se clasifica según dos ejes:

	Temeraria	Prudente
Deliberada	«No tenemos tiempo, entregamos ahora»	«Lo haremos rápido y mejoraremos después»
Inadvertida	«¿Qué componentes tiene esto?»	«Ahora sabemos cómo debería haberse hecho»

5.2. Estrategias de Prueba

5.2.1. Verificación vs. Validación

Verificación	Validación
¿Construimos bien el producto?	¿Construimos el producto correcto ?
Garantía de implementación correcta	Cumplimiento de requisitos del cliente
Incluye pruebas, revisiones, simulaciones	Incluye pruebas de aceptación y prototipos

5.2.2. Malas prácticas

- Suponer que hay partes que no se pueden probar.
- Que el desarrollador no haga pruebas.
- Aislar al equipo de pruebas.
- Involucrar a los testers solo al final.

5.2.3. Estrategias de prueba (pirámide de pruebas)

Martin Fowler propone priorizar así:

1. Pruebas unitarias (muchas, automáticas).
2. Pruebas de integración.
3. Pruebas de interfaz/aceptación (pocas, más caras de automatizar).

5.2.4. Dimensiones de la prueba

Dimensión	Subdimensión	Descripción
Tipo	Funcional / No funcional	Comprobación de funciones / parámetros como rendimiento
Granularidad	Unitaria / Integración / Validación	Nivel del sistema probado
Alcance	Progresión / Regresión / Smoke / Sanity	Cobertura funcional
Ejecución	Manual / Asistida / Automática	Nivel de automatización
Metodología	Guiada / Exploratoria	Nivel de formalización

5.2.5. Pruebas específicas

- **Unitarias: Caja blanca.** Son las pruebas más simples y frecuentes (sobre código individual). Uso de *stubs* para módulos dependientes. Ejemplos: métodos, condiciones de frontera, errores.
- **Integración: Caja gris.** Pruebas de cómo interactúan los componentes. Importante la integración incremental (mayor control de errores). Puede ser ascendente o descendente.
- **Validación (aceptación): Caja negra.** Realistas y orientadas al usuario final. Difíciles de automatizar. Usadas en fases alfa y beta.
- **No funcionales:** Rendimiento, seguridad, recuperación, esfuerzo, despliegue, etc.

5.3. Administración de la Configuración

Definición

Conjunto de actividades para gestionar los cambios durante el ciclo de vida del software, garantizando:

- Trazabilidad
- Control de versiones
- Información actualizada

Se diferencia del mantenimiento en que este aplica cambios, y la administración de configuración controla y registra dichos cambios.

Conceptos clave

- **Elemento de configuración (EC / ICS):** Unidad identificable que debe ser controlada (código, documentación, etc.).
- **Base de datos de configuración (BCD):** Lugar donde se almacenan los EC y sus versiones.
- **Línea base:** Versión estable y acordada de un conjunto de EC.
- **Cambio:** Modificación propuesta a un EC.
- **Control de cambios:** Proceso de evaluar y aprobar cambios.

5.3.1. Ítems de Configuración (IC)

Definición

Cada uno de los elementos que comprenden toda la información producida como parte del proceso de software.

- Programas de cómputo
- Productos de trabajo
- Contenido

5.3.2. Sistema de Administración

- **Elementos componentes:** Herramientas que permiten el acceso y gestión de cada ítem de configuración del software.
- **Elementos de proceso:** Acciones y tareas necesarias para realizar una gestión efectiva del cambio.
- **Elementos de construcción:** Herramientas que automatizan la compilación, empaquetado y generación de versiones correctas del software.
- **Elementos humanos:** Procesos y herramientas que utiliza el equipo para implementar de manera efectiva la administración de configuración.

5.3.3. Repositorio ACS

Definición

Almacén centralizado o distribuido donde se gestionan los ítems de configuración.

Características clave:

- Control de versiones.
- Rastreo de dependencias entre componentes.
- Trazabilidad de requisitos y cambios.
- Apoyo a auditorías e inspecciones de calidad.

5.3.4. Proceso de Administración de la Configuración

1. **Identificación:** Asignar nombres unívocos a los ítems de configuración.
2. **Control de cambios:** Registrar, aprobar e implementar modificaciones.
3. **Control de versiones:** Gestionar diferentes versiones de todos los objetos.
4. **Auditoría:** Verificar que los cambios cumplen con los estándares establecidos.
5. **Reporte de estado:** Informar del estado actual y del histórico de cambios.

5.3.5. Administración del contenido

- **Subsistema de recopilación:** Permite almacenar y organizar el contenido asociado a cada Ítem de Configuración del Software (ICS).
- **Subsistema de administración:** Gestiona los cambios realizados sobre el contenido de cada ICS, manteniendo un histórico completo y trazable.
- **Subsistema de publicación:** Hace accesible el contenido relevante a los distintos interesados del proyecto, permitiendo su consulta o reutilización.

5.3.6. Conflictos dominantes

- **Contenido:** No está claro qué información debe formar parte del sistema de administración de configuración.
- **Personas:** El equipo desconoce las herramientas o procesos de ACS, o no sabe cuándo aplicarlos.
- **Escalabilidad:** A medida que crece el proyecto, se complica la gestión del gran volumen de cambios.
- **Políticas:** Faltan reglas claras sobre quién es responsable de llevar a cabo la administración de configuración y cómo aplicarla de manera coherente.

Capítulo 6

Mantenimiento

Índice general

6.1. Métricas de producto	49
6.1.1. Conceptos clave	49
6.1.2. Principios de medición	49
6.1.3. Atributos de métricas	50
6.1.4. Métricas específicas	50
6.2. Tipos de mantenimiento	52
6.2.1. Leyes de Lehman	52
6.2.2. Proceso de mantenimiento	52
6.2.3. Clasificación de mantenimiento	53
6.2.4. Release Notes	53
6.2.5. Sistemas heredados	53
6.2.6. Reingeniería de sistemas	53
6.2.7. Tipos de reingeniería	53
6.2.8. Flujo de reingeniería	54

6.1. Métricas de producto

Definición

Métrica: Medida cuantitativa del grado en que un sistema, componente o proceso posee un atributo determinado (IEEE Standard).

6.1.1. Conceptos clave

- **Medida:** Un único dato.
- **Medición:** Recolección de varias medidas.
- **Indicador:** Combinación de métricas que proporciona comprensión sobre procesos, proyectos o productos.

6.1.2. Principios de medición

1. **Formulación:** Derivación de medidas apropiadas
2. **Recolección:** Mecanismo para acumular datos

3. **Análisis:** Cálculo de métricas con herramientas matemáticas
4. **Interpretación:** Evaluación de resultados
5. **Retroalimentación:** Recomendaciones al equipo

6.1.3. Atributos de métricas

- Medible
- Intuitiva
- Objetiva
- Coherente
- Tecnológicamente agnóstica
- Accionable

6.1.4. Métricas específicas

Definición

Los puntos de definición son una forma de medir el tamaño, la complejidad y la calidad del software.

- **Modelo de requisitos:**

$$PF = \text{conteo} \cdot (0,65 + (0,01 \cdot \text{FAV}))$$

- Componentes:

Entradas externas (EE) : Información que se origina de un usuario o se transmite desde otra aplicación. Se usan para actualizar archivos lógicos internos (ALI). Las entradas deben distinguirse de las consultas.

Salidas externas (SE) : Datos derivados dentro de la aplicación que ofrecen información al usuario.

Consultas externas (CE) : Entrada que da como resultado la generación de alguna respuesta (con frecuencia recuperada de un ALI).

Número de archivos lógicos internos (ALI) : Agrupamiento lógico de datos que reside dentro de la frontera de la aplicación y se mantiene mediante entradas externas

Número de archivos de interfaz externos (AIE) : Agrupamiento lógico de datos que reside fuera de la aplicación, pero que proporciona información que puede usar la aplicación.

Factor de ajuste de valor (FAV) : Indica la complejidad del sistema en su conjunto en base a unas preguntas a las que se asigna un valor entre 0 (irrelevante) y 5 (esencial).

- **Diseño arquitectónico:**

- Complejidad de módulo: $S(i) = f_{out}^2(i)$, $D(i) = \frac{v(i)}{f_{out}(i)+1}$ (Estructural (S) y Datos (D))
- Complejidad de sistema: $C(i) = S(i) + D(i)$

Donde:

- $f_{out}(i)$ es el número de módulos que dependen del módulo i .
- $v(i)$ es el número de módulos de los que depende el módulo i .
- Tamaño = $n + a$; n es el número de nodos y a es el número de arcos.
- Profundidad: trayectoria más larga desde el nodo raíz hasta un nodo hoja
- Ancho: número máximo de nodos en cualquier nivel de la arquitectura
- **Orientadas a clase (CK):**

Cuadro 6.1: Métricas de la Suite CK

Acrónimo	Descripción
MPC	Métodos Ponderados por Clase: Suma de las complejidades nominales de todos los métodos de una clase ($MPC = \sum c_i$), donde c_i es la complejidad nominal de cada método.
PAH	Profundidad del Árbol de Herencia: Longitud máxima desde el nodo de la clase actual hasta la raíz del árbol de herencia.
NDH	Número de Hijos Directos: Cantidad de subclases que heredan directamente de la clase actual (subclases inmediatas).
FCOM	Falta de Cohesión en Métodos: Número de pares de métodos que acceden a uno o más atributos de clase en común.

- **Código fuente, métricas de Halstead:**

Cuadro 6.2: Métricas de Halstead

Símbolo	Descripción	Fórmula
n1	Número de operadores únicos en un programa	Cantidad de tipos diferentes de operadores
n2	Número de operandos únicos en un programa	Cantidad de tipos diferentes de operandos
N1	Frecuencia total de operadores	$N1 = \sum(\text{ocurrencias de cada operador})$
N2	Frecuencia total de operandos	$N2 = \sum(\text{ocurrencias de cada operando})$

- Longitud: $N = n_1 \log_2 n_1 + n_2 \log_2 n_2$

Nota personal

(Coa estimación de N anterior. Se se usa $N = N_1 + N_2$, o volume sería real.)

- Volumen: $V = N \log_2(n_1 + n_2)$

- **Pruebas:**

- Nivel de programa: $PL = 1/((n_1/2) \cdot (N_2/n_2))$

- Esfuerzo de prueba: $e = V/PL$
- **Mantenimiento:**
 - Índice de madurez: $IMS = (M_1 - (F_a + F_c + F_d))/M_1$
- **SLA/SLO/SLI:**
 - SLA: Acuerdo con cliente (Ejemplo: disponibilidad ¿99.9 %)
 - SLO: Objetivo específico dentro de SLA
 - SLI: Medida real de cumplimiento

6.2. Tipos de mantenimiento

6.2.1. Leyes de Lehman

- **Ley del cambio continuo.** En un entorno real, un sistema debe necesariamente cambiar para mantener su utilidad.
- **Ley de complejidad creciente.** Cuando el sistema evoluciona se hace más complejo. Hay que tomar medidas para evitarlo.
- **Ley de evolución.** La evolución es un proceso autorregulado. El tamaño, tiempo entre versiones, errores detectados, etc., se mantienen en el tiempo.
- **Ley de estabilidad organizacional.** Durante el tiempo de vida del sistema su velocidad de desarrollo es constante e independiente de los recursos dedicados su desarrollo.
- **Ley de conservación de la familiaridad.** A medida que un sistema evoluciona todo lo que está asociado con ello debe mantener un conocimiento total de su contenido y su comportamiento.
- **Ley de crecimiento continuado.** La funcionalidad ofrecida por los sistemas tiene que crecer continuamente para mantener la satisfacción de los usuarios.
- **Ley de decremento de la calidad.** La calidad de los sistemas software comenzará a disminuir a menos que dichos sistemas se adapten a los cambios de su entorno de funcionamiento.
- **Ley de retroalimentación.** Los procesos de evolución incorporan sistemas de retroalimentación.

6.2.2. Proceso de mantenimiento

- **Causas de modificación:**
 - Nuevos requisitos/cambios solicitados
 - Corrección de errores
- **Incorporación al desarrollo:**
 - Implementación formal u hotfixes
- **Factores de esfuerzo:**
 - Diseño del sistema
 - Mecanismos de prueba

- Documentación
- Estabilidad del personal

6.2.3. Clasificación de mantenimiento

Categoría	Tipo	Descripción	Esfuerzo
Evolutivo	Perfectivo	Añadir nuevas funcionalidades	50 %
	Adaptativo	Adaptar a nuevos entornos	25 %
	Preventivo	Mejorar mantenibilidad futura	5 %
Tradicional	Correctivo	Corregir errores	20 %

6.2.4. Release Notes

- **Contenido esencial:**
 - Novedades
 - Mejoras
 - Correcciones de errores

6.2.5. Sistemas heredados

- **Problemas comunes:**
 - Código espagueti
 - Falta de documentación
 - Estructura deficiente
 - Especificaciones ausentes
- **Solución:** Ingeniería inversa

6.2.6. Reingeniería de sistemas

Definición

Reestructuración, reescritura o re-documentación sin cambiar funcionalidad.

- **Ventajas:** Más económico que desarrollo nuevo • Reemplazo gradual
- **Factores coste:** Personal experto, herramientas disponibles
- **¿Por qué?:** Más barato que el desarrollo.

6.2.7. Tipos de reingeniería

- Traducción de código
- Ingeniería inversa
- Reestructuración
- Ingeniería hacia adelante
- Migración de datos

6.2.8. Flujo de reingeniería

1. Código fuente sucio $\rightarrow$ Reestructuración $\rightarrow$ Código limpio
2. Extracción de abstracciones $\rightarrow$ Especificación inicial
3. Refinamiento $\rightarrow$ Especificación final

Anexos

Anexo 1: Clasificación de procesos según ISO/IEC 15504

Nivel	Criterios (checklist)
0 – Incompleto	☐ No se sigue un proceso claro o estructurado. ☐ No se puede saber si se alcanzan los objetivos del proceso.
1 – Realizado	☐ El proceso se realiza de manera consistente. ☐ Los resultados son medibles.
2 – Gestionado	☐ Las actividades están planificadas y controladas. ☐ Los productos de trabajo (artefactos) están controlados.
3 – Establecido	☐ El proceso está documentado y definido. ☐ Está basado en estándares o modelos.
4 – Predecible	☐ Se recogen datos cuantitativos (tempos, esfuerzo, etcétera...). ☐ Se establecen objetivos medibles en base a esos datos.
5 – Optimizado	☐ Hay mejora continua del proceso. ☐ Se identifican puntos débiles y buenas prácticas. ☐ Se implementan cambios activamente para mejorar el proceso.

Cuadro 6.3: Checklist para clasificación de procesos segundo ISO/IEC 15504

Anexo 2: Explicación de la Administración de Configuración de Software (ACS)

¿Qué es ACS?

La Administración de Configuración de Software (ACS) es un conjunto de **procesos y herramientas** para gestionar los **cambios** en productos software durante todo su ciclo de vida.

Va más allá de controlar solo versiones, integrando **documentación, dependencias, auditorías y trazabilidad**.

Diferencias con Git

- **Alcance mayor:** Incluye código, documentación, requisitos, configuraciones...
- **Procesos integrados:** Control de flujos y aprobaciones.
- **Trazabilidad:** Desde requisitos hasta entregables.

Beneficios

- Mejora la calidad y estabilidad.
- Facilita el trabajo en equipos grandes.
- Reduce errores por cambios no controlados.
- Permite auditorías y cumplimiento.

Componentes principales

- **Control de versiones:** Histórico y gestión de versiones.
- **Gestión de dependencias:** Control de relaciones entre componentes.
- **Gestión de cambios:** Registro, planificación y aprobación.
- **Auditoría y trazabilidad:** Seguimiento detallado de cambios.

¿Por dónde empezar?

- Usa un sistema de control de versiones (ej. Git).
- Documenta los elementos de configuración.
- Define políticas y procesos claros.
- Usa herramientas de seguimiento (p.ej. Jira).
- Forma al equipo en buenas prácticas.

Este anexo es una introducción visual y resumida para entender la Administración de Configuración de Software.